zkLLM: Zero Knowledge Proofs for Large Language Models

Haochen Sun
haochen.sun@uwaterloo.ca
University of Waterloo
Waterloo, Ontario, Canada

Jason Li
j2643li@uwaterloo.ca
University of Waterloo
Waterloo, Ontario, Canada

Hongyang Zhang
hongyang.zhang@uwaterloo.ca
University of Waterloo
Waterloo, Ontario, Canada

ABSTRACT

The recent surge in artificial intelligence (AI), characterized by the prominence of large language models (LLMs), has ushered in fundamental transformations across the globe. However, alongside these advancements, concerns surrounding the legitimacy of LLMs have grown, posing legal challenges to their extensive applications. Compounding these concerns, the parameters of LLMs are often treated as intellectual property, restricting direct investigations.

In this study, we address a fundamental challenge within the realm of AI legislation: the need to establish the authenticity of outputs generated by LLMs. To tackle this issue, we present **zkLLM**, which stands as the inaugural specialized zero-knowledge proof tailored for LLMs to the best of our knowledge. Addressing the persistent challenge of non-arithmetic operations in deep learning, we introduce tlookup, a parallelized lookup argument designed for non-arithmetic tensor operations in deep learning, offering a solution with no asymptotic overhead. Furthermore, leveraging the foundation of tlookup, we introduce zkAttn, a specialized zero-knowledge proof crafted for the attention mechanism, carefully balancing considerations of running time, memory usage, and accuracy.

Empowered by our fully parallelized CUDA implementation, zkLLM emerges as a significant stride towards achieving efficient zero-knowledge verifiable computations over LLMs. Remarkably, for LLMs boasting 13 billion parameters, our approach enables the generation of a correctness proof for the entire inference process in under 15 minutes. The resulting proof, compactly sized at less than 200 kB, is designed to uphold the privacy of the model parameters, ensuring no inadvertent information leakage.

CCS CONCEPTS

• **Computing methodologies** → **Machine learning**; • **Security and privacy** → **Cryptography**.

KEYWORDS

large language models; zero-knowledge proofs

ACM Reference Format:
Haochen Sun, Jason Li, and Hongyang Zhang. 2024. zkLLM: Zero Knowledge Proofs for Large Language Models. In *Proceedings of the 2024 ACM SIGSAC Conference on Computer and Communications Security (CCS '24), October 14–18, 2024, Salt Lake City, UT, USA.* ACM, New York, NY, USA, 15 pages. https://doi.org/10.1145/3658644.3670334

CCS '24, October 14–18, 2024, Salt Lake City, UT, USA

ACM ISBN 979-8-4007-0636-3/24/10.
https://doi.org/10.1145/3658644.3670334

1 INTRODUCTION

The recent surge in artificial intelligence (AI), particularly with the advent of Large Language Models (LLMs) [1, 8, 11, 32, 42, 43], has profoundly transformed the world. However, these technological advances have also raised concerns about the legitimacy of these groundbreaking models, challenging the legal underpinnings of their extensive applications. For instance, in December 2023, the New York Times filed a lawsuit against OpenAI and Microsoft, accusing them of using copyrighted material from the newspaper to train their chatbots. In October 2023, President Biden issued an executive order to address both the "myriad benefits" and "substantial risks" posed by AI. As laws and regulations around LLMs evolve and tighten, developing practical tools to verify the legitimacy of these models has become crucial.

Consider the auditing process of a newly-released LLM, which is hosted on a cloud service (e.g., Microsoft Azure) with API access. Law enforcement queries the model using designated prompts to test if the LLM generates illegal output (e.g., untrue, violence-prompting, or racist). In the stringent legal context, the authenticity of the output must be established to exclude the possibility of cheating by manipulating the generated texts. On the other hand, although the architectures are typically described in technical reports, the trained parameters are concealed as the AI developers' intellectual properties, making direct examination of the model parameters impossible. This dilemma calls for the application of zero-knowledge proofs (ZKPs), which allow for verifiable computations over the neural networks while disclosing no information about the neural network parameters [15, 26, 27, 29, 46–48].

However, adapting existing ZKP techniques to modern LLMs, characterized by their immense scale, presents significant challenges. These models require substantial computational resources, which general-purpose ZKP frameworks [3–5, 7, 9, 19, 23, 28, 33], often unaware of LLM structure and limited in parallel computation support, struggle to provide.

While early research has explored specialized cryptographic protocols for specific neural network architectures like convolutional neural networks (CNNs) [27, 29, 48], LLMs' complex internal structures necessitate further innovation in ZKP protocol design. This innovation is vital to avoid the excessive overhead typical of general-purpose ZKPs. LLMs involve many non-arithmetic operations, such as GELU [25] and SwiGLU [38] activation functions, which only partially align with current ZKP methods. Lookup arguments, trading memory consumption for faster runtimes, have been introduced [26] to handle these nonlinearities, but their straightforward application raises questions about manageable memory overhead.

Moreover, the attention mechanism in LLMs [44], which is inherently multivariate and often employs the Softmax function, requires a tailored ZKP protocol design for effective management of proof overhead. Tackling this mechanism within ZKPs is challenging, particularly as its components are not typically found in previously explored neural network architectures, such as MLPs and CNNs. In these traditional models, Softmax functions are usually placed after the output layer and are therefore not considered in prior works on zero-knowledge verifiable deep learning. This setup is in stark contrast with LLMs, where Softmax functions are used extensively across multiple layers. This prevalent use in LLMs necessitates a more refined approach in ZKP design to ensure both precise and efficient zero-knowledge verification, especially given the unique challenges presented by the attention mechanism.

In response to these challenges, we present **zkLLM**, the inaugural ZKP scheme specifically designed for LLMs. zkLLM empowers LLM owners to validate the integrity of inference outcomes to stakeholders, such as law enforcement agencies, thereby streamlining investigations involving LLMs while safeguarding intellectual property. Our key contributions are:

- We propose tlookup, a unique ZKP protocol for universal non-arithmetic operations in deep learning, to tackle the persistent challenge of verifying such operations (e.g., activation functions). tlookup adeptly handles overhead in two ways: analytically, it adds no asymptotic overhead in memory complexity or running time; practically, its design promotes a high level of parallelization, fully leveraging parallel computing resources (like GPUs) commonly used in LLM computing environments.
- We introduce zkAttn, a ZKP specifically crafted for attention mechanisms in LLMs. Building upon tlookup and enhancing its capabilities, zkAttn mitigates the accuracy degradation and high overheads linked with bit-decompositions and polynomial approximations. It also removes the necessity to list all multivariate input-output pairs, a prerequisite in lookup-based methods, by harnessing the mathematical properties of the attention mechanism. This strategy strikes a balance between running time, memory usage, and accuracy, while maintaining security and privacy standards.
- Our efficient CUDA implementation, in conjunction with the aforementioned technical advancements, positions zkLLM as the trailblazing ZKP for LLMs of sizes up to 13 billion parameters. zkLLM achieves reasonable proving times of 1-15 minutes and produces compact proofs smaller than 200kB. These proofs can be verified within 1-3 seconds by the verifier and guarantee no exposure of model parameters.

2 TECHNICAL OVERVIEW

Compared with general-purpose counterparts, an efficient zero-knowledge proof system specialized for deep learning hinges critically upon two key requirements:

- The capability for extensive parallelization (for example, using CUDA), which allows for the handling of proofs for the entire computational process in a reasonable timeframe.
- The adept handling of non-arithmetic operations, encompassing activation functions among others.

Although sumcheck-based protocols are known to be compatible with tensor structures common in deep learning computations [21, 29] , traditionally, they have depended on bit-decomposition methods for non-arithmetic operations. This dependence leads to an increase in prover overhead and restricts the variety of non-arithmetic operations that can be supported. In response, we have developed a novel sumcheck-based protocol for lookup arguments over tensors.

Our design capitalizes on the following fact: for $\mathbf{S} \in \mathbb{F}^D$ and $\mathbf{T} \in \mathbb{F}^N$, the set inclusion $\mathbf{S} \subseteq \mathbf{T}$ holds if and only if there is an $\mathbf{m}$ such that $\sum_{i\in[D]}(X+\mathbf{S}_i)^{-1} \equiv \sum_{i\in[N]} \mathbf{m}_i(X+\mathbf{T}_i)^{-1}$ as rational functions over $X \in \mathbb{F}$ [24]. This equivalence can be verified by evaluating both expressions at a single point $X \leftarrow \beta$, randomly selected by the verifier. Furthermore, $\mathbf{m}$ can be computed in $O(D)$ time using straightforward counting. Hence, by calculating the elementwise multiplicative inversions $\mathbf{A} \leftarrow (\beta + \mathbf{S})^{-1}$ and $\mathbf{B} \leftarrow (\beta + \mathbf{T})^{-1}$, we can parallelize the sumcheck protocol for the identity

$$(\mathbf{A}.\text{sum}() = \mathbf{m}^\top \mathbf{B}) \wedge (\mathbf{A} \odot (\beta + \mathbf{S}) = \mathbf{1}) \wedge (\mathbf{B} \odot (\beta + \mathbf{T}) = \mathbf{1}) \quad (1)$$

This approach stands in contrast to the sequential lookup arguments [13, 17, 18, 35, 52, 53] that are based on univariate polynomials.

For the attention mechanism widely applied in modern LLMs, represented by the equation

$$\text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) := \text{Softmax}\left(\frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{d}}\right)\mathbf{V}, \quad (2)$$

the direct application of lookup arguments, such as tlookup, presents the impractical challenge of compiling all input-output pairs into a lookup table due to the multivariate nature of the attention mechanism. To achieve zero-knowledge verifiability with limited overhead for the attention mechanism, we introduce zkAttn, as depicted in Figure 1:

(1) Implement the matrix multiplication between the query $\mathbf{Q}$ and keys $\mathbf{K}^\top$, resulting in $\mathbf{Z} \leftarrow \mathbf{Q}\mathbf{K}^\top$. This process is verifiable through the dedicated sumcheck protocol designed specifically for matrix multiplications.
(2) Exploit the shift-invariance property of Softmax to adjust each row of $\mathbf{Z}$ by a constant, represented as a vector $\hat{\mathbf{z}}$, so that $\exp(\mathbf{Z} - \hat{\mathbf{z}}\mathbf{1}^\top)$ sums to 1 row-wise. This transformation renders the Softmax output equivalent to applying $\exp(\cdot)$ element-wise to $\mathbf{Z}' := \mathbf{Z} - \hat{\mathbf{z}}\mathbf{1}^\top$. However, computing $\hat{\mathbf{z}}$ from $\mathbf{Z}$ is highly intricate and not directly verifiable.
(3) Transform $\mathbf{Z}'$ into negative K-digit base-b numbers, with each $\mathbf{Z}' = -\sum_{k=0}^{K-1} b^k \mathbf{Z}^{(k)}$. By utilizing the homomorphism of $\exp(\cdot)$, the Softmax output $\mathbf{Y}$ is then expressed as

$$\mathbf{Y} = \exp\left(-\sum_{k=0}^{K-1} b^k \mathbf{Z}^{(k)}\right) = \prod_{k=0}^{K-1} \exp\left(-b^k \mathbf{Z}^{(k)}\right), \quad (3)$$

with a tlookup installed for each term of the K in the product to handle the non-arithmetic operation.
(4) Rather than verifying the correctness of $\hat{\mathbf{z}}$ directly, which is highly non-arithmetic, an additional check is introduced to ensure the rowwise sums of $\mathbf{Y}$ equal 1.
(5) Implement another verifiable matrix multiplication between the Softmax output $\mathbf{Y}$ and the values $\mathbf{V}$.

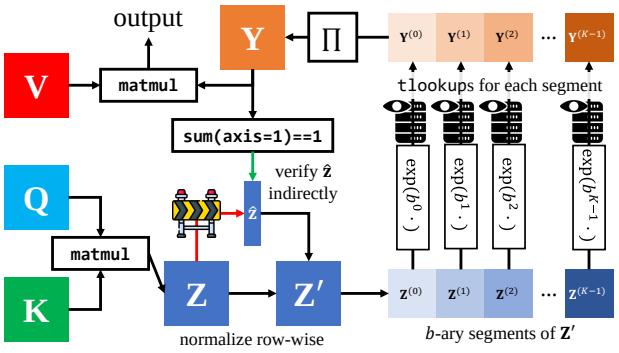

Figure 1: Overview of zkAttn **for** (2).

Note that the above overview omits details about the handling of scaling factors and quantization errors for clarity. The design of zkAttn manages the overhead of verifiable computation for the highly non-arithmetic operations within the attention mechanism while preserving computational accuracy.

3 PRELIMINARIES

3.1 Notations

We represent vectors and tensors in bold font, such as $\mathbf{v}$ for vectors and $\mathbf{S}$ for tensors. For clarity of the analysis and consistency with the cryptographic frameworks we utilize, we apply 0-based indexing to all mathematical structures. For simple operations and indexing over tensors, we adhere to the PyTorch conventions, using notations like $\mathbf{v}_{[i]}$ or the more concise $\mathbf{v}_i$ for elements, $\mathbf{S}_{[i,j_0:j_1,:]}$ for slicing, and $\mathbf{S}$.sum(axis=0) for aggregation along a specified dimension. Moreover, we denote the set of non-negative integers less than a positive integer N using the compact form $[N]$, which signifies $\{0, 1, \ldots, N-1\}$.

3.2 Large language models, transformers and the attention mechanism

Large Language Models (LLMs), exemplified by the GPT series [8, 32], PaLM [11], and LLaMa [42], are renowned for their exceptional performance in general-purpose language understanding and generation tasks. These models are built upon the transformer architecture, which is fundamentally based on the attention mechanism [44].

As depicted in Figure 3, LLMs typically consist of multiple layers that transform the embeddings of an input sequence of tokens using multi-head attention. In each attention head i, parameterized by linear weights $\mathbf{W}_i^Q$, $\mathbf{W}_i^K$, and $\mathbf{W}_i^V$, the queries $\mathbf{Q}_i \leftarrow \mathbf{X}\mathbf{W}_i^Q$, keys $\mathbf{K}_i \leftarrow \mathbf{X}\mathbf{W}_i^K$, and values $\mathbf{V}_i \leftarrow \mathbf{X}\mathbf{W}_i^V$ are computed. These components are then processed by the Attention function, concatenated, and projected using another linear weight $\mathbf{W}^O$:

$$\mathbf{O} \leftarrow \text{Concat}_i\left(\text{Attention}\left(\mathbf{Q}_i, \mathbf{K}_i, \mathbf{V}_i\right)\right)\mathbf{W}^O, \tag{4}$$

The output $\mathbf{O}$ is subsequently processed by a feed-forward multilayer perceptron (MLP). The activations from the final layer are then transformed into a probability distribution over the output tokens, from which the output sequence is autoregressively sampled. The

Q: As the developer of an LLM, law enforcement seeks to verify the model's authenticity by having me execute the inference process using their designated prompts. I aim to establish the authenticity of my output without revealing any details about the model parameters. Can ZKP be utilized in the inference process?
A: Yes, ZKP can be utilized in the inference process of an LLM to establish the authenticity of the output without revealing any details about the model parameters. This allows one party to prove to another that a certain statement is true, without conveying any additional information. In this case, it can prove that the model's output came from a specific model.
Q: What are some benefits of applying ZKPs in this scenario?
A: Utilizing ZKPs in the inference process of an LLM can provide authenticity and privacy. It can confirm the output's origin from a specific model without revealing any model details, protecting any sensitive or proprietary information. The generated proofs are verifiable, allowing anyone to confirm the output's authenticity. Some ZKP protocols are also scalable, accommodating large models and complex computations, which is beneficial for LLMs.

Figure 2: An example dialogue with GPT-3.5 regarding zkLLM's motivation

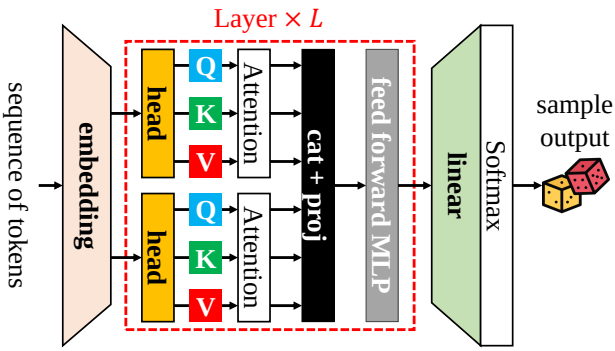

Figure 3: Typical structure of LLMs

Attention function, as defined in (2), effectively mimics cognitive attention and has significantly contributed to the success of LLMs. The adaptation of the attention mechanism for zero-knowledge verifiability is a primary focus of this study.

3.3 Sumcheck protocol, multilinear extensions and tensor operations

The correctness of arithmetic tensor operations (e.g., matrix multiplication) is verified using the *sumcheck protocol* [12] over the *multilinear extensions* [30] of the tensors involved.

Consider a tensor $\mathbf{S} \in \mathbb{F}^{D_0 \times D_1 \times \cdots \times D_{K-1}}$ discretized into a finite field $\mathbb{F}$ via scaling and rounding. Without loss of generality, assume that D_ks are all powers of 2 for $0 \le k \le K-1$, or zero-padding may be applied. Thus, writing indices in binary format, $\mathbf{S}$

can also be considered as a function $\mathrm{S}(\cdot) : \{0,1\}^{\sum_{k=0}^{K-1}\log_2 D_k} \to \mathbb{F}$. Here, $\mathrm{S}(\mathbf{i}_0, \mathbf{i}_1, \ldots, \mathbf{i}_{K-1}) = \mathrm{S}_{[i_0, i_1, \ldots, i_{K-1}]}$ where $\mathbf{i}_k$ is the binary representation of any $0 \le i_k \le D_{K-1} - 1$. A multivariate polynomial $\widetilde{\mathrm{S}}(\cdot) : \mathbb{F}^{\sum_{k=0}^{K-1}\log_2 D_k} \to \mathbb{F}$ is a *multilinear extension* of $\mathrm{S}(\cdot)$ such that $\widetilde{\mathrm{S}}(\cdot) \equiv \mathrm{S}(\cdot)$ on $\{0,1\}^{\sum_{k=0}^{K-1}\log_2 D_k}$, practically implemented as

$$\widetilde{\mathrm{S}}(\mathbf{u}_0, \mathbf{u}_1, \ldots, \mathbf{u}_{K-1}) = \sum_{\substack{\mathbf{i}_k \in \{0,1\}^{\log_2 D_k} \\ 0 \le k \le K-1}} \widetilde{\mathbf{e}}\left(\bigoplus_{k=0}^{K-1} \mathbf{u}_k, \bigoplus_{k=0}^{K-1} \mathbf{i}_k\right) \mathrm{S}(\mathbf{i}_0, \mathbf{i}_1, \ldots, \mathbf{i}_{K-1}), \quad (5)$$

where $\widetilde{\mathbf{e}}(\mathbf{u}, \mathbf{v}) := \sum_{i=0}^{d-1} \mathbf{u}_i \mathbf{v}_i + (1-\mathbf{u}_i)(1-\mathbf{v}_i)$ for any d-dimensional $\mathbf{u}, \mathbf{v} \in \mathbb{F}^d$, which reduces to the equality indicator $\mathbb{1}_{\{\mathbf{u}=\mathbf{v}\}}$ when restricted to $\mathbf{u}, \mathbf{v} \in \{0,1\}^d$.

The correctness of a tensor operation can be expressed as equalities over the tensors. For instance, for matrix multiplication $\mathbf{C} \leftarrow \mathbf{AB}$, where $\mathbf{C} \in \mathbb{F}^{D_0 \times D_2}$, $\mathbf{A} \in \mathbb{F}^{D_0 \times D_1}$, and $\mathbf{B} \in \mathbb{F}^{D_1 \times D_2}$, the correctness is characterized by $\mathbf{C}_{[i,j]} = \sum_{k=0}^{D_1-1} \mathbf{A}_{[i,k]} \mathbf{B}_{[k,j]}$ for each i, j, or equivalently,

$$\sum_{\mathbf{k} \in \{0,1\}^{\log_2 D_1}} \left(D_1^{-1} \widetilde{\mathbf{C}}(\mathbf{i}, \mathbf{j}) - \widetilde{\mathbf{A}}(\mathbf{i}, \mathbf{k}) \widetilde{\mathbf{B}}(\mathbf{k}, \mathbf{j}) \right) = 0. \quad (6)$$

By applying the Schwartz-Zippel Lemma [37, 59], with high probability, (6) holds for all $\mathbf{i}, \mathbf{j}$ if and only if the random linear combination

$$\sum_{\substack{\mathbf{i} \in \{0,1\}^{\log_2 D_0} \\ \mathbf{j} \in \{0,1\}^{\log_2 D_2} \\ \mathbf{k} \in \{0,1\}^{\log_2 D_1}}} \widetilde{\mathbf{e}}(\mathbf{u}_0 \oplus \mathbf{u}_2, \mathbf{i} \oplus \mathbf{j}) \underbrace{\left(D_1^{-1} \widetilde{\mathbf{C}}(\mathbf{i}, \mathbf{j}) - \widetilde{\mathbf{A}}(\mathbf{i}, \mathbf{k}) \widetilde{\mathbf{B}}(\mathbf{k}, \mathbf{j}) \right)}_{\text{varies for other tensor operations}} = 0, \quad (7)$$

where $\widetilde{\mathbf{e}}(\cdot)$. Thus, the prover and the verifier can execute the sumcheck protocol [12], which proves the statements in the form of

$$\sum_{\mathbf{i} \in \{0,1\}^d} f(\mathbf{i}) = 0 \quad (8)$$

for any d-variate polynomial ($d = \log_2 D_0 + \log_2 D_1 + \log_2 D_2$ in the case of (7)). The prover time, proof size, and verifier time are $O(2^d)$, $O(d)$, and $O(d)$, respectively. At the end of the protocol, a claim about the value of $f(\mathbf{v})$ is made by the prover (where $\mathbf{v} \sim \mathbb{F}^d$ due to the randomness over the protocol execution), which is further reduced to the claimed evaluations of the multilinear extensions (i.e., $\widetilde{\mathbf{C}}(\mathbf{v}_0, \mathbf{v}_2)$, $\widetilde{\mathbf{A}}(\mathbf{v}_0, \mathbf{v}_1)$, $\widetilde{\mathbf{B}}(\mathbf{v}_1, \mathbf{v}_2)$ in (7) with the indices decomposed as $\mathbf{v} = \mathbf{v}_0 \oplus \mathbf{v}_1 \oplus \mathbf{v}_2$ with the corresponding dimensionalities.) These claims are further verified via the proof of evaluations on the commitments of the tensors introduced in Section 3.4.

Optimized adaptations of the sumcheck protocol are designed to align with standard operations in deep learning, such as matrix multiplication [21, 40] (see Section 6.1.1) and convolution [29]. The preservation of the tensor structure enables the parallelization of the proof. Moreover, zero-knowledge adaptations of the sumcheck protocol [10, 50, 51] have been developed to prevent any disclosure of information related to the tensors, while adding a negligible additional computational burden.

3.4 Polynomial Commitment

The binding and hiding requirements for the tensors, in the form of multilinear extensions, which are considered the intellectual properties of the prover, are achieved using polynomial commitment schemes. Specifically, the following establishes the correctness of $\widetilde{\mathrm{S}}(\mathbf{v})$ in zero-knowledge for any tensor S (assumed to be one-dimensional for simplicity) and any $\mathbf{v}$ with matching dimensionality:

- $\mathsf{pp} \leftarrow \mathsf{KeyGen}(1^\lambda)$ generates the public parameters used in the scheme, where λ is the security parameter of the scheme.
- $[\![\mathrm{S}]\!] \leftarrow \mathsf{Commit}(\mathrm{S}, r; \mathsf{pp})$ generates a binding and hiding commitment $[\![\mathrm{S}]\!]$ of S, such that $[\![\mathrm{S}]\!]$ leaks no information about S, and no polynomial-time adversary can compute $\mathrm{S}' \neq \mathrm{S}$ and r' such that $[\![\mathrm{S}]\!] = \mathsf{Commit}(\mathrm{S}; \mathsf{pp}, r)$.
- $(y, \pi) \leftarrow \mathsf{ProveEval}(\mathrm{S}, [\![\mathrm{S}]\!], \mathbf{v}, r; \mathsf{pp})$ allows the prover to compute $y \leftarrow \widetilde{\mathrm{S}}(\mathbf{v})$ for any $\mathbf{v}$ with matching dimensionality, and creates a *proof of evaluation* that $y = \widetilde{\mathrm{S}}(\mathbf{v})$ with respect to the committed S.
- $\mathsf{True/False} \leftarrow \mathsf{Verify}(y, \pi, [\![\mathrm{S}]\!], \mathbf{v}; \mathsf{pp})$ allows the verifier to verify the correctness of y, such that
  - **(Completeness)** if $(y, \pi) = \mathsf{ProveEval}(\mathrm{S}, [\![\mathrm{S}]\!], \mathbf{v}, r; \mathsf{pp})$, then the output is True.
  - **(Soundness)** if $y \neq \widetilde{\mathrm{S}}(\mathbf{v})$, then the output is False with $1 - \mathrm{negl}(\lambda)$ probability.
  - **(Zero-knowledge)** the verifier learns no information beyond $y = \widetilde{\mathrm{S}}(\mathbf{v})$.

In the absence of ambiguity, we omit the randomness r and public parameters pp in the subsequent context.

In this study, Hyrax [45], a variant of the Pedersen commitment [34] that does not require a trusted setup, is used as an instantiation of the polynomial commitment scheme. It operates on a cyclic group $\mathbb{G}$ (typically an elliptic curve), with the hardness assumption of the discrete log problem, and is isomorphic to the addition of $\mathbb{F}$. Hyrax is homomorphic, such that $\mathsf{Commit}(\mathrm{S}_1, r_1) + \mathsf{Commit}(\mathrm{S}_2, r_2) = \mathsf{Commit}(\mathrm{S}_1 + \mathrm{S}_2, r_1 + r_2)$ for any two tensors $\mathrm{S}_1, \mathrm{S}_2$ and randomness r_1, r_2. Hyrax achieves linear complexity in Commit and ProveEval with respect to the dimensionality D of the tensor S involved and can be further parallelized by concurrently handling operations on all dimensions. It also balances the commitment size, proof size, and verifier's proof evaluation time, all to sub-linear complexities of $O(\sqrt{D})$, $O(\log D)$, and $O(\sqrt{D})$ respectively. These improvements are adeptly adopted in zkLLM to minimize both computation and communication burdens.

3.5 Lookup arguments

The lookup argument is commonly used to address non-arithmetic operations within the domain of zero-knowledge proofs [41]. Inspired by recent advances [13, 17, 18, 35, 52, 53], the incorporation of lookup arguments into zero-knowledge verifiable deep learning inference [26] has been pursued. In such a setting, a lookup argument verifies that each element in a secret tensor S^D, known only to the prover, is contained within a predefined table $\mathbf{T} \in \mathbb{F}^N$, mutually acknowledged by both parties. However, the requisite computations

for lookup arguments are intrinsically sequential, which contradicts the parallelism preferred in deep learning environments. Furthermore, deploying lookup arguments entails a trade-off between sacrificing precision and incurring excessive memory consumption and trusted setup burdens due to the expansive size of the lookup tables necessary to cover all possible values (with N being significantly large).

In response to these challenges, our proposed lookup arguments for non-arithmetic tensor operations markedly enhance parallelization compared to the largely sequential approaches traditionally used in verifiable deep learning inference. Additionally, our novel proof protocol, tailored for the Softmax function within the attention mechanisms of Transformer models, is designed to optimize the balance between setup and proving times, memory consumptions, and precision.

3.6 Settings and security assumptions

We follow the widely recognized framework for zero-knowledge verifiable inferences as outlined in prior research on zero-knowledge machine learning [15, 26, 27, 29, 46–48]. In this framework, the **prover** (such as an AI company) owns an LLM with a publicly known structure (e.g., described in the technical report), while considering the model's weights as its intellectual property. The prover provides API access to this model for a **verifier** (like an AI regulation enforcer), who submits a prompt and requests formal proof that the inference result returned by the API is accurate in relation to the prompt and the confidential model.

A semi-honest assumption is applied to the verifier: the verifier accurately reports the outcome of the proof verification (whether it is accepted or rejected) but endeavors to glean additional information about the LLM (like hidden parameters) beyond merely confirming the correctness of the inference result.

In this study, we assume the use of a commitment scheme that ensures λ-bit security. Correspondingly, the computations are carried out within a finite field $\mathbb{F}$, characterized by a prime order of at least $\Omega(2^{2\lambda})$. Furthermore, we postulate that every aspect of the transformer model and the data—including the number of layers, the dimensions of tensors, and the complexity of operations between them—is polynomially bounded by λ.

4 tlookup: VERIFIABLE NON-ARITHMETIC OPERATIONS FOR DEEP LEARNING

In this section, we introduce tlookup, our novel approach to addressing general non-arithmetic operations in deep learning. The tlookup design preserves the widely-used tensor-based structure, guaranteeing seamless compatibility with the established computational frameworks in deep learning. tlookup acts as a foundational component of zkAttn, our specialized ZKP protocol tailored for the attention mechanism, detailed in Section 5. Furthermore, tlookup is applicable to other non-arithmetic operations essential to the inference mechanisms within LLMs.

We first reduce the non-arithmetic tensor operations to lookup arguments over tensors. Specifically, for a tensor $\mathbf{S} \in \mathbb{F}^D$, the prover aims to convince the verifier that each element of $\mathbf{S}$ exists within $\mathbf{T} \in \mathbb{F}^N$, a table that both parties have full knowledge of. The essence of our approach hinges on the subsequent lemma:

Lemma 4.1 ([24]). *Given tensors $\mathbf{S} \in \mathbb{F}^D$ and $\mathbf{T} \in \mathbb{F}^N$, $\mathbf{S} \subset \mathbf{T}$ as sets if and only if there exists $\mathbf{m} \in \mathbb{F}^N$ such that the following identity of rational functions is satisfied:*

$$\sum_{i\in[D]} \frac{1}{X+\mathbf{S}_i} = \sum_{i\in[N]} \frac{\mathbf{m}_i}{X+\mathbf{T}_i}. \tag{9}$$

When the condition $\mathbf{S} \subset \mathbf{T}$ holds, the prover constructs $\mathbf{m}$ as:

$$\mathbf{m}_i \leftarrow \left|\left\{j : \mathbf{S}_j = \mathbf{T}_i\right\}\right|, \text{ for } 0 \le i \le N-1. \tag{10}$$

The verifier can then confirm the equality presented in (9) by randomly choosing $X \leftarrow \beta \sim \mathbb{F}$. By defining:

$$\mathbf{A} := \left(\frac{1}{\beta+\mathbf{S}_i}\right)_{i=0}^{D-1}, \mathbf{B} := \left(\frac{1}{\beta+\mathbf{T}_i}\right)_{i=0}^{N-1}, \tag{11}$$

the aforementioned equality at the random point β can be restated as:

$$\sum_{i\in[D]} \mathbf{A}_i = \sum_{i\in[N]} \mathbf{m}_i \mathbf{B}_i. \tag{12}$$

Therefore, with the randomness $\mathbf{u} \sim \mathbb{F}^{\log_2 D}$ and $\alpha \sim \mathbb{F}$, the sum-check for the correctness of (11) and (12) can be formulated as

$$\begin{aligned} 0 =& \left(\sum_{\mathbf{i}\in[D]} \widetilde{\mathbf{A}}(\mathbf{i}) - \sum_{\mathbf{j}\in[N]} \widetilde{\mathbf{m}}(\mathbf{j})\widetilde{\mathbf{B}}(\mathbf{j})\right) \\ &+ \alpha\left(\sum_{\mathbf{i}\in[D]} \widetilde{\mathbf{e}}(\mathbf{u},\mathbf{i})\widetilde{\mathbf{A}}(\mathbf{i})\left(\widetilde{\mathbf{S}}(\mathbf{i})+\beta\right) - 1\right) \\ &+ \alpha^2\left(\sum_{\mathbf{j}\in[N]} \widetilde{\mathbf{e}}\left(\mathbf{u}_{\left[\log_2 \frac{D}{N}:\right]},\mathbf{j}\right)\widetilde{\mathbf{B}}(\mathbf{j})\left(\widetilde{\mathbf{T}}(\mathbf{j})+\beta\right) - 1\right), \end{aligned} \tag{13}$$

or equivalently,

$$\begin{aligned} \alpha+\alpha^2 = \sum_{\mathbf{i}\in\left[\frac{D}{N}\right]}\sum_{\mathbf{j}\in[N]} &\left(\widetilde{\mathbf{A}}(\mathbf{i}\oplus\mathbf{j})\left(\alpha\widetilde{\mathbf{e}}(\mathbf{u},\mathbf{i}\oplus\mathbf{j})\left(\widetilde{\mathbf{S}}(\mathbf{i}\oplus\mathbf{j})+\beta\right)+1\right)\right. \\ &\left. + ND^{-1}\widetilde{\mathbf{B}}(\mathbf{j})\left(\alpha^2\widetilde{\mathbf{e}}\left(\mathbf{u}_{\left[\log_2 \frac{D}{N}:\right]},\mathbf{j}\right)\left(\widetilde{\mathbf{T}}(\mathbf{j})+\beta\right) - \widetilde{\mathbf{m}}(\mathbf{j})\right)\right). \end{aligned} \tag{14}$$

A comprehensive description of the procedure to validate $\mathbf{S} \subset \mathbf{T}$ is found in Protocol 1. In particular, in Line 1, tlookup-Setup($\mathbf{T}$) generates a short witness $[\![\mathbf{T}]\!]$ to a prescribed table $\mathbf{T}$ known to both parties; in Line 4, the prover constructs $\mathbf{m}$ based on a tensor $\mathbf{S}$ and table $\mathbf{T}$ and commit to $\mathbf{S}$ and $\mathbf{m}$ using tlookup-Prep($\mathbf{S},\mathbf{T}$); finally, in Line 9, $\langle\mathcal{P},\mathcal{V}\rangle$.tlookup-Prove($[\![\mathbf{S}]\!],[\![\mathbf{m}]\!],[\![\mathbf{T}]\!]$) is the interactive process of the prover $\mathcal{P}$ proving that a secret tensor $\mathbf{S}$ is elementwisely in $\mathbf{T}$, which has been committed as $[\![\mathbf{T}]\!]$.

Meanwhile, for elementwise non-arithmetic operations $f : \mathcal{X} \to \mathcal{Y}$ over tensors where $\mathcal{X},\mathcal{Y} \subset \mathbb{F}$, two lookup tables can be constructed: $\mathbf{T}_\mathcal{X} := (x)_{x\in\mathcal{X}}$ and $\mathbf{T}_\mathcal{Y} := (f(x))_{x\in\mathcal{X}}$. To demonstrate that $\mathbf{Y} = f(\mathbf{X})$ for some $\mathbf{X},\mathbf{Y} \in \mathbb{F}^D$ (broadcasting f over all dimensions), one can apply the idea of random linear combination to reduce the check to one instance of Protocol 1 where $\mathbf{X}+\alpha\mathbf{Y} \subset \mathbf{T}_\mathcal{X} + \alpha\mathbf{T}_\mathcal{Y}$ for $\alpha \sim \mathbb{F}$ chosen by the verifier.

Example 4.2 (ReLU with rescaling). *We first consider the rectified linear unit (ReLU), which is a common activation function in contemporary deep learning models, including Transformers. ReLUs are generally applied subsequent to linear layers (for instance, fully*

Protocol 1 tlookup

Require: The prover $\mathcal{P}$ knows $\mathbf{S} \in \mathbb{F}^D$. N, D are both powers of 2 such that N divides D.

procedure TLOOKUP-SETUP($\mathbf{T} \in \mathbb{F}^N$)
  return $[\![\mathbf{T}]\!] \leftarrow \mathsf{Commit}(\mathbf{T}; 0)$ ▷ No hiding required
end procedure
procedure $\mathcal{P}$.TLOOKUP-PREP($\mathbf{S} \in \mathbb{F}^D, \mathbf{T} \in \mathbb{F}^N$)
  Compute $\mathbf{m} = \mathbf{m}(\mathbf{S}, \mathbf{T})$ as (10)
  $\mathcal{P} \to \mathcal{V} : [\![\mathbf{S}]\!] \leftarrow \mathsf{Commit}(\mathbf{S})$
  $\mathcal{P} \to \mathcal{V} : [\![\mathbf{m}]\!] \leftarrow \mathsf{Commit}(\mathbf{m})$
end procedure
procedure $\langle \mathcal{P}, \mathcal{V} \rangle$.TLOOKUP-PROVE($[\![\mathbf{S}]\!], [\![\mathbf{m}]\!], [\![\mathbf{T}]\!]$)
  $\mathcal{V} \to \mathcal{P} : \beta \sim \mathbb{F}$
  $\mathcal{P}$ computes $\mathbf{A}, \mathbf{B}$ as (11)
  $\mathcal{P} \to \mathcal{V} : [\![\mathbf{A}]\!] \leftarrow \mathsf{Commit}(\mathbf{A}), [\![\mathbf{B}]\!] \leftarrow \mathsf{Commit}(\mathbf{B})$
  $\mathcal{P}$ and $\mathcal{V}$ run the sumcheck on (14), followed by the proofs of evaluation on $[\![\mathbf{A}]\!], [\![\mathbf{B}]\!], [\![\mathbf{S}]\!], [\![\mathbf{m}]\!]$ and $[\![\mathbf{T}]\!]$.
end procedure

connected layers) where products are involved. In the scenario of fully quantized computation, it becomes necessary for the ReLU to incorporate rescaling as well. This is denoted as follows:

$$\mathbf{A} \leftarrow \mathit{ReLU}(\mathbf{Z}) = \left\lfloor \frac{\mathbf{Z}}{\gamma} \right\rfloor \odot \mathbb{1}\left\{ \left\lfloor \frac{\mathbf{Z}}{\gamma} \right\rfloor \geq 0 \right\}, \tag{15}$$

where γ represents the scaling factor used in the system (assumed to be even for simplicity). We assume that $-\frac{B}{2} \leq \left\lfloor \frac{\mathbf{Z}}{\gamma} \right\rfloor < \frac{B}{2}$ holds element-wise for a positive even integer B. Considering that $\mathbf{Z}$ is decomposed as $\mathbf{Z}' := \left\lfloor \frac{\mathbf{Z}}{\gamma} \right\rfloor$ and $\mathbf{R} = \mathbf{Z} - \gamma\mathbf{Z}'$, we establish a pair of input-output lookup tables for $\mathbf{Z}'$. These are defined as $\mathbf{T}_{\mathcal{X}} := \left[-\frac{B}{2}, \frac{B}{2} - 1\right]$ and $\mathbf{T}_{\mathcal{Y}} := \mathbf{T}^+_{\mathcal{X}}$ (i.e., taking the maximum with 0 element-wise), and an additional lookup table for $\mathbf{R}$ as $\mathbf{T}_{\mathcal{R}} = \left[-\frac{\gamma}{2}, \frac{\gamma}{2} - 1\right]$. By requiring the prover to demonstrate to the verifier that $\mathbf{Z}' + \alpha\mathbf{A} \subset \mathbf{T}_{\mathcal{X}} + \alpha\mathbf{T}_{\mathcal{Y}}$ for a random α, and that $\mathbf{R} \subset \mathbf{T}_{\mathcal{R}}$, both using Protocol 1, in addition to proving the decomposition as $\mathbf{Z} = \gamma\mathbf{Z}' + \mathbf{R}$, we can sufficiently validate the correctness of inference through the ReLU function. Notably, unlike the brute-force method that employs a single lookup table and incurs an $O(B\gamma)$ overhead in both running time and memory usage, the use of two lookup tables effectively reduces this overhead to $O(B + \gamma)$. Similarly, if γ is too large to fit the table into memory, it can be further divided into a K-digit $\gamma^{\frac{1}{K}}$-ary number. In this scenario, each of the K digits in the remainder corresponds to a separate tlookup*, thus adequately covering all possible values of the remainder.*

However, resolving the long-standing problem of excessive memory consumption for lookup tables in the realm of deep learning requires additional efforts. Specifically, in Section 5, tlookup is further refined into zkAttn to address its multivariate and highly non-arithmetic nature, optimizing the balance among running time, memory consumption, and approximation error.

5 zkAttn: DEDICATED ZKP FOR THE ATTENTION MECHANISM IN LLMS

The attention mechanism is a key component in modern transformers, including state-of-the-art LLMs. However, incorporating these mechanisms into ZKP backends has been challenging, primarily due to their distinctive mathematical properties. Specifically, the Softmax function, integral to the attention mechanism, involves non-arithmetic operations like exponentiation, and its multivariate aspect complicates the use of polynomial approximations for traditional ZKP backends. To address these challenges, we introduce zkAttn, a specialized ZKP tailored for the attention mechanism, designed to leverage its inherent mathematical characteristics effectively.

5.1 Formulation of zkAttn

The attention mechanism, in its discretized form, accepts as input a value matrix $\mathbf{V} \in \mathbb{F}^{n\times d}$, a key matrix $\mathbf{K} \in \mathbb{F}^{n\times d}$, and a query matrix $\mathbf{Q} \in \mathbb{F}^{m\times d}$. It produces the output $\mathrm{Softmax}\left(\frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{d}}\right)\mathbf{V}$ subject to appropriate rescaling of the input and output due to quantization, where Softmax is applied row-wise. In this discussion, we focus on $\mathrm{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \mathrm{Softmax}\left(\frac{\mathbf{Z}}{\sqrt{d}}\right)$, where the input matrix $\mathbf{Z} = \mathbf{Q}\mathbf{K}^\top$ is presumed to be scaled by the scaling factor γ from its actual values. It is assumed that d is a constant, known to both the prover and verifier, stemming from the presumption of a known model architecture. Equivalently, for each row $\mathbf{z} = (z_0, z_1, \ldots, z_{n-1}) \in \mathbb{F}^n$, the objective is to devise an algorithm that computes

$$s(\mathbf{z}) := \left(\frac{\exp\left(\frac{z_i}{\gamma\sqrt{d}}\right)}{\sum_{j=0}^{n-1} \exp\left(\frac{z_j}{\gamma\sqrt{d}}\right)} \right)_{i=0}^{n-1} \tag{16}$$

in the real domain. Alternatively, it should compute its quantized counterpart $\theta s(\mathbf{z})$, ensuring limited numerical error and manageable proof generation overhead. Here, θ represents the scaling factor of all Softmax outputs in the system. Notably, this factor differs from the scaling factor γ used for other matrices, such as $\mathbf{Q}$, $\mathbf{K}$, and $\mathbf{V}$. For the sake of streamlined and verifiable rescaling in subsequent computations, we posit that θ is a multiple of γ.

To circumvent the verification of real division operations—which can lead to remainders after quantization—we observe the following. By utilizing the shift-invariance property of Softmax and defining

$$\hat{z} := \gamma\sqrt{d} \ln\left(\sum_{j=0}^{n-1} \exp\left(\frac{z_j}{\gamma\sqrt{d}}\right) \right), \tag{17}$$

we derive that

$$s(\mathbf{z}) = s(\mathbf{z} - \hat{z}) = \left(\exp\left(\frac{z_i - \hat{z}}{\gamma\sqrt{d}}\right) \right)_{i=0}^{n-1}. \tag{18}$$

It is imperative to understand that the computation of $\hat{z}$ ($\lfloor \hat{z} \rceil$ to be specific, since $\hat{z}$ is not an integer in general) in (17) is not directly verified due to its highly non-arithmetic nature. Instead, the prover ensures that the output of $s(\mathbf{z})$ adheres to proper normalization. In its quantized representation, the sum of its dimensions must equal θ. A certain degree of deviation is acceptable owing to quantization, and the precise bounds of this error will be elucidated in Section 7.1. Furthermore, beyond verifying normalization, there exists the challenge of crafting a scheme to compute the quantized exponentiation. This scheme should not only be accurate, approximating

$\theta \exp\left(\frac{\cdot}{\gamma\sqrt{d}}\right)$ with minimal error, but also be amenable to efficient verification through the proof protocol that will be subsequently introduced.

Observe that, given the definition of $\hat{z}$, $z_i - \hat{z} \leq 0$ for all is. On the other hand, as z_is are all real numbers involved in the matrix multiplication scaled by γ, it is also reasonable to assume that each $z_i - \hat{z}$ is lower bounded by some integer $-B$ such that $\gamma \ll B \ll |\mathbb{F}|$, such that $(-B, 0]$ can accommodate the reasonable values of $z_i - \hat{z}$, but sufficiently small so as not to cause wraparounds in $\mathbb{F}$.

However, it may still be prohibitive to install a single tlookup to accommodate all B possible input values to achieve the verifiable exp operation, due to the enormous size of B. Therefore, we instead install K tlookups, with sizes $b^{(0)}, b^{(1)}, \ldots, b^{(K-1)}$ each, such that $B = \prod_{k=0}^{K-1} b^{(k)}$, to accommodate entire input space of $(-B, 0]$. Specifically, by defining $B^{(k)}$ as

$$B^{(k)} := \begin{cases} 1, & \text{if } k = 0; \\ \prod_{j=0}^{k-1} b^{(j)}, & \text{if } 1 \leq k \leq K-1 \end{cases},$$

a bijection $\mathfrak{b} : \prod_{k=0}^{K-1} \left[b^{(k)}\right] \to (-B, 0]$ can be expressed as

$$\mathfrak{b}\left(x^{(0)}, x^{(1)}, \ldots, x^{(K-1)}\right) = -\sum_{k=0}^{K-1} x^{(k)} B^{(k)}, \tag{19}$$

where each $x^{(k)} \in \left[b^{(k)}\right] := \left\{0, 1, ..., b^{(k)} - 1\right\}$ corresponds to the k-th tlookup. Note that, in the event that all $b^{(k)}$s equal to the same number b, Equation (19) is equivalent to the b-arization.

Consequently, for $\left(x^{(0)}, x^{(1)}, \ldots, x^{(K-1)}\right) = \mathfrak{b}^{-1}(z)$ where $z \in (-B, 0]$, the following holds:

$$\exp\left(\frac{z}{\gamma\sqrt{d}}\right) = \exp\left(-\frac{\sum_{k=0}^{K-1} x^{(k)} B^{(k)}}{\gamma\sqrt{d}}\right) = \prod_{k=0}^{K-1} \exp\left(-\frac{B^{(k)}}{\gamma\sqrt{d}} x^{(k)}\right). \tag{20}$$

Our objective is to compute the quantized representation of equation (20), taking into account the scaling factor θ. If we further decompose θ as $\theta = \prod_{k=0}^{K-1} \theta^{(k)}$ with non-negative values for $\theta^{(k)}$, equation (20) gives rise to

$$\theta \exp\left(\frac{z}{\gamma\sqrt{d}}\right) = \prod_{k=0}^{K-1} \theta^{(k)} \exp\left(-\frac{B^{(k)}}{\gamma\sqrt{d}} x^{(k)}\right). \tag{21}$$

Following the decomposition in Equation (21), we can construct K tlookup tables $\mathbf{T}^{(k)} = \left(\mathbf{T}^{(k)}_{\mathcal{X}}, \mathbf{T}^{(k)}_{\mathcal{Y}}\right)$. Each table $\mathbf{T}^{(k)}$ comprises all potential input-output pairs corresponding to the k-th term in the product of (21):

$$\mathbf{T}^{(k)}_{\mathcal{X}} := \left[b^{(k)}\right], \quad \mathbf{T}^{(k)}_{\mathcal{Y}} := \left(\left\lfloor \theta^{(k)} \exp\left(-\frac{B^{(k)}}{\gamma\sqrt{d}} x\right)\right\rfloor\right)_{x \in \left[b^{(k)}\right]}. \tag{22}$$

Given any input $z \in (-B, 0]$, the prover first decomposes $\mathfrak{b}^{-1}(z) = \left(x^{(0)}, x^{(1)}, \ldots, x^{(K-1)}\right)$ according to (19). Each component $x^{(k)}$ is subsequently mapped to $y^{(k)}$ based on $\mathbf{T}^{(k)}$, resulting in the computation $y \leftarrow \prod_{k=0}^{K-1} y^{(k)}$. Subsequently, the prover must demonstrate to the verifier that:

$$z + \sum_{k=0}^{K-1} x^{(k)} B^{(k)} = 0, \tag{23}$$

$$\left(x^{(k)}, y^{(k)}\right) \in \mathbf{T}^{(k)}, \qquad \forall 0 \leq k \leq K-1, \tag{24}$$

$$\prod_{k=0}^{K-1} y^{(k)} = y. \tag{25}$$

Equation (23) confirms that the decomposition via $\mathfrak{b}^{-1}$ is valid. Equation (24) ensures the correctness of the exponent in each component concerning the pre-computed values in $\mathbf{T}^{(k)}$, and (25) asserts that the output y is accurately derived using the homomorphism of exponentiation from each factor $y^{(k)}$. Together, these three conditions guarantee the correct computation of the exponentiation operation, up to the rounding errors. Specifically:

Lemma 5.1. *Conditions* (23), (24), *and* (25) *imply:*

$$y = \prod_{k=0}^{K-1} \left\lfloor \theta^{(k)} \exp\left(-\frac{B^{(k)}}{\gamma\sqrt{d}} x^{(k)}\right)\right\rfloor, \tag{26}$$

where $\left(x^{(k)}\right)_{k=0}^{K-1} = \mathfrak{b}^{-1}(z)$ *is the valid decomposition according to* (19).

The deviation between y and the exact scaled exponent

$$\theta \exp\left(\frac{z}{\gamma\sqrt{d}}\right) = \prod_{k=0}^{K-1} \theta^{(k)} \exp\left(-\frac{B^{(k)}}{\gamma\sqrt{d}} x^{(k)}\right)$$

arises only from the rounding of each factor. An in-depth analysis of this will be covered in Section 7.1, where we will also provide guidance on selecting the parameters $\theta^{(k)}$ and $B^{(k)}$. In the subsequent sections of this text, we delve into the protocol design, facilitating the batched verification of (23), (24), and (25) for each dimension of large tensors used in transformer computations.

5.1.1 Optimization for the most and least significant segments. For the uppermost significant M segments, specifically $x^{(k)}$ with $K - M \leq k \leq K - 1$, consider a scenario where if any of these segments $x^{(k)}$ have non-zero values, the resulting exponent

$$\exp\left(\frac{z}{\gamma\sqrt{d}}\right) \leq \exp\left(-\frac{B^{(K-M)}}{\gamma\sqrt{d}}\right) \tag{27}$$

approximates 0 closely enough that the output y from (25) can be designated as 0. This outcome can be achieved by configuring each table $\mathbf{T}^{(k)}$ that $K - M \leq k \leq K - 1$ in (24), to yield $y^{(k)} = 0$ for any $x^{(k)} > 0$. Moreover, based on our initial design, instances where $x^{(k)} = 0$, the value of $y^{(k)}$ defaults to $\left\lfloor \theta^{(k)} \right\rfloor$. Clearly, assigning any value other than 1 to these $\theta^{(k)}$ would only amplify the errors in $\mathbf{T}^{(k)}$ and other tables, especially under the constraint that $\prod_{k=0}^{K-1} \theta^{(k)} = \theta$ is constant. Therefore, for these most significant M segments, the lookup tables $\mathbf{T}^{(k)}$ can be reduced to the indicator function $y^{(k)} = \mathbb{1}\{x^{(k)} = 0\}$.

On the other hand, for the least significant L segments $x^{(k)}$, indexed by $0 \leq k \leq L - 1$, the expression $\exp\left(-\frac{B^{(k)}}{\gamma\sqrt{d}} x^{(k)}\right)$ tends

to hover close to 1 for all possible values of $0 \le x^{(k)} \le b^{(k)} - 1$. Given this, approximating the exponentiation as a constant of 1 incurs a negligible error. Analogous to the strategy for the most significant segments, it is efficient to set the scaling factors $\theta^{(k)}$ to 1, sidestepping larger alternatives. This approach frees up room for allocating larger $\theta^{(k)}$s for segments indexed by $L \le k \le K - M - 1$, thereby enhancing precision for these segments. As a result, the constraint (24) for validating input-output pairs simplifies to $x^{(k)} \in \left[b^{(k)}\right]$, given that $y^{(k)}$ consistently equates to 1.

As delineated in Section 7.1, employing these optimizations for both the most and least significant segments tightly upper bounds the error in zkAttn, aligning closely with the computational logic of the original neural networks.

5.2 zkAttn: the main protocol

In Protocol 2, we present the technical details of zkAttn built upon tlookup, our protocol for general non-arithmetic operations in deep learning. zkAttn is separated into three steps. First, zkAttn-Setup(·) (in Line 1) completes the setup for zkAttn by generating short witnesses of all tables involved. Then, in zkAttn-Compute(·), the prover is responsible for computing the output of the attention mechanism and all necessary auxiliary tensors (e.g., the normalization constants, the inputs and outputs of each segment, and the recovered row-wise sum that should not excessively deviate from 1) for the zero-knowledge verifiability of the attention mechanism, and sends the commitments of these tensors to the verifier. Finally, the prover and verifier engage in the interactive protocol zkAttn-Prove(·) in Line 14, which involves proving the correctness of each segment and the normalization using the specialized lookup arguments of tlookup, as well as all arithmetic relations connecting the auxiliary tensors.

6 PUTTING EVERYTHING TOGETHER

6.1 Taxonomy of verifiable tensor operations

In this section, we present a taxonomy of the tensor operations involved in modern LLMs and the customized handling of these operations by zkLLM.

6.1.1 Matrix multiplications. Matrix multiplication plays a crucial role in modern transformers, including all linear layers and positional encoding (e.g., RoPE [39]) in LLMs.

Dedicated sumchecks [21] for matrix multiplications have achieved running times significantly lower than the computation itself. This development has been a key driver in the creation of specialized ZKPs for deep learning, a method also applied in this study to establish the correctness of all matrix products. To confirm $\mathbf{C} = \mathbf{AB}$, with $\mathbf{A} \in \mathbb{F}^{m \times n}$ and $\mathbf{B} \in \mathbb{F}^{n \times p}$, the prover and the verifier execute a sumcheck on

$$\widetilde{\mathbf{C}}(\mathbf{u}, \mathbf{v}) = \sum_{\mathbf{i} \in \{0,1\}^{\lceil \log_2 n \rceil}} \widetilde{\mathbf{A}}(\mathbf{u}, \mathbf{i})\, \widetilde{\mathbf{B}}(\mathbf{i}, \mathbf{v}), \tag{28}$$

where $\mathbf{u} \in \mathbb{F}^{\lceil \log_2 m \rceil}$ and $\mathbf{v} \in \mathbb{F}^{\lceil \log_2 p \rceil}$ are selected at random by the verifier. This specialized proof for matrix multiplication ensures a prover time of $O(mn + np)$, faster than the computation process.

6.1.2 Activation functions. To enhance performance, modern LLMs have replaced traditional ReLU activation functions with smoother alternatives like SwiGLU [38] and GELU [25]. This transition necessitates extra efforts to make these new activation functions verifiable. The SwiGLU function, parameterized by β, is defined as

$$\text{SwiGLU}_\beta(z, z') := \text{Swish}_\beta(z) \cdot z', \tag{29}$$

with the Swish function being $\text{Swish}_\beta(z) := z \cdot \text{Sigmoid}(\beta z)$. Though the non-arithmetic Sigmoid function can be integrated into the proof system via tlookup, optimizing setup costs and memory usage is crucial. This is achieved by reducing the Sigmoid function to zkAttn, given $\text{Softmax}\begin{pmatrix} z \\ 0 \end{pmatrix} = \begin{pmatrix} \text{Sigmoid}(z) \\ \text{Sigmoid}(-z) \end{pmatrix}$, thus circumventing the bottleneck of iterating over extensive input-output pairs. The GELU function, defined as $\text{GELU}(z) := z\Phi(z) \approx z\text{Sigmoid}(1.702z)$, is handled similarly.

6.1.3 Normalization. LLMs employ LayerNorm [2] and its variants (e.g., RMSNorm [54]) for training stability. Unlike batch normalization, which can be merged into preceding layers for verifiable inference, LayerNorm involves non-linear transformations within each sample $\mathbf{x}$, described as

$$y \leftarrow \frac{\mathbf{x} - \mathbb{E}[\mathbf{x}]}{\sqrt{\text{Var}[x] + \epsilon}}. \tag{30}$$

The compound non-arithmetic operations of square-root and inverse are managed through two sequential tlookup steps. These steps are responsible for the verifiable downscaling of the input and the quantized compound operation, respectively. Similar to the ReLU example presented in Example 4.2, the implementation of the first tlookup for downscaling is designed to reduce the overall sizes of the lookup tables, thereby keeping memory usage at a reasonable level.

6.2 Assembly of the proofs

Following the pioneering works [21, 29], zkLLM utilizes sumcheck-based proofs for computations across various components of LLMs. These proofs are assembled in reverse logical order of the arithmetic circuits, methodically reducing the claimed multilinear extension values of the output y to those associated with the prompt $\mathbf{X}$ and the model parameters $\mathbf{W}$. These are then verified straightforwardly and through proof of evaluations on the commitment $[\![\mathbf{W}]\!]$. For high-level clarity, zkLLM can be distilled into three main components, with additional commitments caused by tlookups omitted for simplicity:

- $[\![\mathbf{W}]\!] \leftarrow$ zkLLM-Commit$(\mathbf{W}, \text{pp}, r)$: The prover commits to the model parameters $\mathbf{W}$ using the public parameters (generators of the commitment scheme) pp and randomness r.
- $(y, \pi) \leftarrow$ zkLLM-Prove$(\mathbf{W}, \mathbf{X}, \text{pp}, r)$: The prover computes the output y with the prompt $\mathbf{X}$ and model $\mathbf{W}$, and assembles the proof π using the sumcheck protocols and proofs of evaluations as previously described.
- $b \leftarrow$ zkLLM-Verify$(\mathbf{X}, y, [\![\mathbf{W}]\!])$: The verifier checks the correctness of each sumcheck and proof of evaluation within π, outputting $b = 1$ to accept the proof (if all components are correctly verified), and $b = 0$ otherwise.

Protocol 2 zkAttn (See Section 6.1.1 about the two matrix multiplications involved)

Require: Both the prover $\mathcal{P}$ and the verifier $\mathcal{V}$ know: the lower bound of input $-B$, and the factorization $B = \prod_{k=0}^{K-1} b^{(k)}$; the number of the most and least significant segments M and L in Section 5.1.1; the scaling factor of the input γ, the output θ, and each segment $\theta^{(k)}$ for each $L \le k \le K-M-1$; the parameters m, n, d related to the dimensions of the input; the tolerable error E in row-wise normalization.

procedure zkAttn-Setup($B, K, M, L, \left(b^{(k)}\right)_{k=0}^{K-1}, \gamma, \theta, \left(\theta^{(k)}\right)_{k=L}^{K-M-1}, m, n, d, E$)
  for $0 \le k \le K-1$ **do**
    $\left[\!\left[\mathbf{T}_{\mathcal{X}}^{(k)}\right]\!\right] \leftarrow$ tlookup-Setup($\mathbf{T}_{\mathcal{X}}^{(k)}$) ▷ $\mathbf{T}_{\mathcal{X}}^{(k)} = \left[b^{(k)}\right]$, i.e., the input of the k-th segment
  end for
  for $L \le k \le K-M-1$ **do**
    $\left[\!\left[\mathbf{T}_{\mathcal{Y}}^{(k)}\right]\!\right] \leftarrow$ tlookup-Setup($\mathbf{T}_{\mathcal{Y}}^{(k)}$) ▷ $\mathbf{T}_{\mathcal{Y}}^{(k)}$ as defined in (22), i.e., the output of the k-th segment
  end for
  for $K-M \le k \le K-1$ **do**
    $\left[\!\left[\mathbf{T}_{\mathcal{Y}}^{(k)}\right]\!\right] \leftarrow$ tlookup-Setup($\mathbf{T}_{\mathcal{Y}}^{(k)}$) ▷ $\mathbf{T}_{\mathcal{Y}}^{(k)} = \mathbb{1}\left\{\left[b^{(k)}\right] = 0\right\}$, i.e., the optimized output of the k-th segment
  end for
  $[\![\mathbf{T}_{\mathcal{R}}]\!] \leftarrow$ tlookup-Setup($\mathbf{T}_{\mathcal{R}}$) ▷ $\mathbf{T}_{\mathcal{R}} = [\theta - E, \theta + E]$, i.e., all tolerable values of row-wise sum of the output
  return $\left(\left[\!\left[\mathbf{T}_{\mathcal{X}}^{(k)}\right]\!\right]\right)_{k=0}^{K-1}, \left(\left[\!\left[\mathbf{T}_{\mathcal{Y}}^{(k)}\right]\!\right]\right)_{k=L}^{K-1}, [\![\mathbf{T}_{\mathcal{R}}]\!]$
end procedure

procedure $\mathcal{P}$.zkAttn-Compute($\mathbf{Z} \in \mathbb{F}^{m \times n}, \left(\mathbf{T}_{\mathcal{X}}^{(k)}\right)_{k=0}^{K-1}, \left(\mathbf{T}_{\mathcal{Y}}^{(k)}\right)_{k=L}^{K-1}$) ▷ Some implicit parameters included in Setup($\cdot$) omitted
  $\mathbf{Z}' \leftarrow \mathbf{Z} - \lfloor \hat{\mathbf{z}} \rceil \mathbf{1}^\top$, where $\hat{\mathbf{z}} \in \mathbb{R}^m$ is computed row-wise as (17)
  $\left(\mathbf{X}^{(k)}\right)_{k=0}^{K-1} \leftarrow \mathfrak{b}^{-1}(\mathbf{Z}')$ ▷ $\mathbf{Z}'$ is decomposed elementwisely
  for $k \leftarrow L, L+1, \ldots, K-1$ **do**
    $\mathbf{Y}^{(k)} \leftarrow f^{(k)}(\mathbf{X}^{(k)})$ elementwisely, where $f^{(k)}$ is defined by $\mathbf{T}_{\mathcal{X}}^{(k)}, \mathbf{T}_{\mathcal{Y}}^{(k)}$
  end for
  $\mathbf{Y} \leftarrow \bigodot_{k=L}^{K-1} \mathbf{Y}^{(k)}$ ▷ Compute the final output
  $\hat{\mathbf{y}} \leftarrow \mathbf{Y}$.sum(axis = 1) ▷ For checking the normalization of each row
  $\mathcal{P} \to \mathcal{V} : [\![\mathbf{Z}]\!], [\![\lfloor \hat{\mathbf{z}} \rceil]\!], \left(\left[\!\left[\mathbf{X}^{(k)}\right]\!\right]\right)_{k=0}^{K-1}, [\![\mathbf{Y}]\!], \left(\left[\!\left[\mathbf{Y}^{(k)}\right]\!\right]\right)_{k=L}^{K-1}, [\![\hat{\mathbf{y}}]\!]$
end procedure

procedure $\langle \mathcal{P}, \mathcal{V} \rangle$.zkAttn-Prove($[\![\mathbf{Z}]\!], [\![\lfloor \hat{\mathbf{z}} \rceil]\!], \left(\left[\!\left[\mathbf{X}^{(k)}\right]\!\right]\right)_{k=0}^{K-1}, [\![\mathbf{Y}]\!], \left(\left[\!\left[\mathbf{Y}^{(k)}\right]\!\right]\right)_{k=L}^{K-1}, [\![\hat{\mathbf{y}}]\!]$)
  for $k \leftarrow 0, 1, \ldots, K-1$ **do**
    $\mathcal{P}$.tlookup-Prep($\mathbf{X}^{(k)}, \mathbf{T}_{\mathcal{X}}^{(k)}$) ▷ $\left[\!\left[\mathbf{m}^{(k)} := \mathbf{m}\left(\mathbf{X}^{(k)}, \mathbf{T}_{\mathcal{X}}^{(k)}\right)\right]\!\right]$ transmitted to $\mathcal{V}$
    $\langle \mathcal{P}, \mathcal{V} \rangle$.tlookup-Prove($\left[\!\left[\mathbf{X}^{(k)}\right]\!\right], \left[\!\left[\mathbf{m}^{(k)}\right]\!\right], \left[\!\left[\mathbf{T}_{\mathcal{X}}^{(k)}\right]\!\right]$) ▷ Prove the correctness on the k-th segment
  end for
  $\mathcal{V} \to \mathcal{P} : \alpha \sim \mathbb{F}$
  for $k \leftarrow L, L+1, \ldots K-1$ **do**
    $\mathcal{P}$.tlookup-Prep($\mathbf{X}^{(k)} + \alpha \mathbf{Y}^{(k)}, \mathbf{T}_{\mathcal{X}}^{(k)} + \alpha \mathbf{T}_{\mathcal{Y}}^{(k)}$) ▷ $\left[\!\left[\mathbf{m}^{(k)} := \mathbf{m}\left(\mathbf{X}^{(k)} + \alpha \mathbf{Y}^{(k)}, \mathbf{T}_{\mathcal{X}}^{(k)} + \alpha \mathbf{T}_{\mathcal{Y}}^{(k)}\right)\right]\!\right]$ transmitted to $\mathcal{V}$
    $\langle \mathcal{P}, \mathcal{V} \rangle$.tlookup-Prove($\left[\!\left[\mathbf{X}^{(k)}\right]\!\right] + \alpha \left[\!\left[\mathbf{Y}^{(k)}\right]\!\right], \left[\!\left[\mathbf{m}^{(k)}\right]\!\right], \left[\!\left[\mathbf{T}_{\mathcal{X}}^{(k)}\right]\!\right] + \alpha \left[\!\left[\mathbf{T}_{\mathcal{Y}}^{(k)}\right]\!\right]$) ▷ Prove the correctness on the k-th segment
  end for
  $\mathcal{P}$.tlookup-Prep($\hat{\mathbf{y}}, \mathbf{T}_{\mathcal{R}}$) ▷ $[\![\mathbf{m}_{\mathcal{R}} := \mathbf{m}(\mathbf{Y}, \mathbf{T}_{\mathcal{R}})]\!]$ transmitted to $\mathcal{V}$
  $\langle \mathcal{P}, \mathcal{V} \rangle$.tlookup-Prove($[\![\hat{\mathbf{y}}]\!], [\![\mathbf{m}_{\mathcal{R}}]\!], [\![\mathbf{T}_{\mathcal{R}}]\!]$)
  $\mathcal{P}$ and $\mathcal{V}$ run the sumcheck for $\mathfrak{b}(\mathbf{X}_0, \mathbf{X}_1, \ldots, \mathbf{X}_{K-1}) = \mathbf{Z}' \wedge \mathbf{Z}' = \mathbf{Z} - \lfloor \hat{\mathbf{z}} \rceil \mathbf{1}^\top \wedge \mathbf{Y} = \bigodot_{k=L}^{K-1} \mathbf{Y}^{(k)} \wedge \hat{\mathbf{y}} = \mathbf{Y}$.sum(axis = 1), followed by the proof of evaluations on $[\![\mathbf{Z}]\!], [\![\lfloor \hat{\mathbf{z}} \rceil]\!], \left(\left[\!\left[\mathbf{X}^{(k)}\right]\!\right]\right)_{k=0}^{K-1}, [\![\mathbf{Y}]\!], \left(\left[\!\left[\mathbf{Y}^{(k)}\right]\!\right]\right)_{k=L}^{K-1}, [\![\hat{\mathbf{y}}]\!]$
end procedure

7 ANALYSIS

7.1 Error analysis on zkAttn

In this section, we examine the error introduced by zkAttn, as discussed in Section 5. Our analysis serves three primary objectives:

first, to establish an upper bound on the error, thereby demonstrating that our zkAttn design remains faithful to the original neural network; second, to fine-tune the parameters integral to zkAttn, aiming to minimize the error; and third, to determine an acceptable upper bound on the error upon the verification of proper normalization in zkAttn.

The overall error of zkAttn originates from two sources, the rounding of the shifting factor $\hat{z}$ in (17) that makes the normalization no longer perfect, and the rounding of each segment encoded in the lookup tables $\mathbf{T}^{(k)}$s that introduces errors to the exponentiations. The bound of the overall error is stated in Theorem 7.1 and analyzed in details in the appendix of the long version.

Theorem 7.1 (Error bound). *With the choice of*

$$B^{(K-M)} \leftarrow \frac{\gamma\sqrt{d}}{K-M-L+1}\left((K-M-L)\ln(2n)+\ln\theta\right), \quad (31)$$

$$\theta^{(k)} \leftarrow \exp\left(\frac{B^{(k)}}{\gamma\sqrt{d}}\left(b^{(k)}-1\right)\right)\left(\theta\exp\left(-\frac{B^{(K-M)}-B^{(L)}}{\gamma\sqrt{d}}\right)\right)^{\frac{1}{K-M-L}} \quad (32)$$

and irrelevant of the choice of other $B^{(k)}$s, the error of zkAttn *can be bounded above by*

$$\varepsilon_{attn} = O\left((K-M-L)\left(\frac{n}{\theta}\right)^{\frac{1}{K-M-L+1}}\right). \quad (33)$$

Theorem 7.1 have multiple implications:

(1) Minimizing $K-M-L$, (i.e., the number of segments that are designated as neither the most significant nor the least significant as in Section 5.1.1) and $B^{(L)}$ (i.e., the magnitude of least significant segments) is in favour of reducing the error. However, as will be discussed in Section 7.3, this incurs an undue increase in the computational overhead due to the sizes of the lookup tables.
(2) $\varepsilon_{\text{attn}}$ has no dependence on the segmentation of the zkAttn input except $B^{(L)}$ and $B^{(K-M)}$, leaving room for distributing the sizes of lookup tables $\mathbf{T}^{(k)}$ evenly which compresses the computational overhead associated.
(3) $\varepsilon_{\text{attn}}$ defines the tolerable error row-wise sum upon checking the normalization, i.e., the sum of each row must lie within $[(1-\varepsilon_{\text{attn}})\theta, (1+\varepsilon_{\text{attn}})\theta]$.

7.2 Security and privacy analysis

In this section, we formalize the security and privacy aspects of zkLLM, focusing on the tlookup protocol which facilitates zero-knowledge verifiable computations across all non-arithmetic components. We first address the completeness error of tlookup:

Theorem 7.2 (Completeness of Protocol 1). *Assuming the verifier $\mathcal{V}$ is semi-honest, Protocol 1 incurs a completeness error of $O\left(\frac{N}{|\mathbb{F}|}\right)$.*

Theorem 7.2 indicates that if the prover $\mathcal{P}$ follows the protocol, the proof produced using Protocol 1 has a mere $O\left(\frac{N}{|\mathbb{F}|}\right)$ chance of being rejected. As detailed in the appendix of the long version, this minor imperfection stems from the probability that the random challenge β, chosen by the verifier in Line 10, might trigger a division-by-zero error in one of the terms on either side of Equation (9). On the other hand, all arithmetic tensor operations do not contribute any additional completeness error, thanks to the direct application of the sumcheck protocol. Therefore, the overall probability of an honest prover failing to convince a semi-honest verifier of the correctness of its inference through the LLM is at most $O\left(\frac{C}{|\mathbb{F}|}\right)$, where C represents the total size of all tensors involved in non-arithmetic operations. Given that the entire complexity of the inference is $\text{poly}(\lambda)$ and $|\mathbb{F}| = \Omega\left(2^{2\lambda}\right)$, this error remains negligible in λ. Practically, as our implementation employs the BLS12-381 curve with $|\mathbb{F}| \approx 2^{254}$, far exceeding the computational limits of current technology, verification has never failed in any experiment conducted, as reported in Section 8.

Similarly, the soundness error of tlookup is also negligible in λ, as stated in Theorem 7.3. Coupled with the direct application of the sumcheck protocol and the proof-of-opening for the committed tensors, which also incur negligible errors in λ due to the polynomial λ assumption on the complexity of the entire inference process, we theoretically establish that a valid proof can confirm the correctness of the inference result except with only a negligible probability.

Theorem 7.3 (Soundness of Protocol 1). *For any probabilistic polynomial-time (p.p.t.) prover $\mathcal{P}$, if in Line 6, the message $\mathcal{P}$ sends to $\mathcal{V}$ is $[\![\mathrm{S}]\!] \leftarrow \mathit{Commit}(\mathrm{S})$ such that $\mathrm{S} \not\subset \mathbf{T}$, then except with probability* $\text{negl}(\lambda)$*, the execution of Protocol 1 is unsuccessful, resulting in the semi-honest verifier $\mathcal{V}$ rejecting the proof.*

Proof sketch of Theorem 7.3. By the binding property of the commitment scheme, except with probability $\text{negl}(\lambda)$, in Line 13, the success of proofs of evaluations implies the correctness of all claimed multilinear extension values on $\mathbf{A}, \mathbf{B}, \mathbf{S}, \mathbf{m}$, and $\mathbf{T}$. Subsequently, the success of the sumchecks implies with $1 - O\left(\frac{D}{|\mathbb{F}|}\right)$ that all equalities in Equations (11) and (12) hold, such that

$$\sum_{i\in[D]} \frac{1}{\beta+\mathbf{S}_i} = \sum_{i\in[N]} \frac{\mathbf{m}_i}{\beta+\mathbf{T}_i}. \quad (34)$$

Finally, given the randomness of β, with probability $1 - O\left(\frac{ND}{|\mathbb{F}|}\right)$, Equation (34) implies Equation (9), leading to the conclusion that $\mathbf{S} \subset \mathbf{T}$. □

It is noteworthy, as elaborated in Section 7.1, that the correctness of zkAttn is quantified by an $\mathcal{L}_1$ error of $\varepsilon_{\text{attn}}$. This measure of correctness similarly applies to other exponentiation-based activation functions. The correctness of all other non-arithmetic operations must also consider the quantization errors. For example, as highlighted in Example 4.2, a numerical error margin of $\frac{1}{2\gamma}$ is an inescapable consequence of the rescaling process. On the other hand, this degree of tolerance is also sufficient, as numerical errors exceeding $\frac{1}{2\gamma}$ would be detectable as incorrect computations and consequently rejected through the application of tlookup.

Finally, with the application of zero-knowledge variations of sumcheck protocols [10, 29, 50, 51] and Pedersen commitment schemes [34], the proof assembled by zkLLM does not disclose any information about the protected model parameters. Formally,

Theorem 7.4 (Zero-knowledge, adapted from [29]). *Assuming the application of zero-knowledge variations of sumcheck protocols [10, 29, 50, 51] and Pedersen commitment schemes [34], there exists a simulator $\mathcal{S} = (\mathcal{S}_1, \mathcal{S}_2)$ such that the following two views are computationally indistinguishable to any probabilistic polynomial-time (PPT) algorithm $\mathcal{A}$, given the public parameters pp (the generators used in the commitment scheme within the context of zkLLM):*

$Real_{\mathcal{A},\mathbf{W}}(pp)$:
1: $[\![\mathbf{W}]\!] \leftarrow$ *zkLLM-Commit*$(\mathbf{W}, pp, r)$
2: $\mathbf{X} \leftarrow \mathcal{A}([\![\mathbf{W}]\!], pp)$
3: $(y, \pi) \leftarrow$ *zkLLM-Prove*$(\mathbf{W}, \mathbf{X}, pp, r)$
4: $b \leftarrow \mathcal{A}([\![\mathbf{W}]\!], \mathbf{X}, y, \pi, pp)$
5: ***return*** b

$Ideal_{\mathcal{A},\mathcal{S}^{\mathcal{A}}}(pp)$:
1: $com \leftarrow \mathcal{S}_1(1^\lambda, pp, r)$
2: $\mathbf{X} \leftarrow \mathcal{A}(com, pp)$
3: $(y, \pi) \leftarrow \mathcal{S}_2^{\mathcal{A}}(com, \mathbf{X}, pp, r)$, *given oracle access to* $y =$ *zkLLM-compute*$(\mathbf{W}, \mathbf{X})$
4: $b \leftarrow \mathcal{A}(com, \mathbf{X}, y, \pi, pp)$
5: ***return*** b

For any PPT algorithm $\mathcal{A}$ and all LLM (represented by the parameter) $\mathbf{W}$, there exists a simulator $\mathcal{S}$ such that

$$\left|\mathbb{P}\left(Real_{\mathcal{A},\mathbf{W}}(pp) = 1\right) - \mathbb{P}\left(Ideal_{\mathcal{A},\mathcal{S}^{\mathcal{A}}}(pp) = 1\right)\right| \leq \mathrm{negl}(\lambda). \quad (35)$$

7.3 Overhead analysis

In this section, we analyze the overhead of zkLLM, focusing on the running times for both the prover and the verifier, as well as the memory and communication costs.

7.3.1 Overhead of tlookup. In Protocol 1, we adhere to the assumption that $N = O(D)$. This guideline is strictly followed in our implementation due to the substantial sizes of tensors involved in LLM computations. The linear time complexity for both committing and proving in the Pedersen commitments and the sumcheck protocols results in a total computational complexity of $O(D)$ for the prover in Protocol 1. Similarly, memory requirements are maintained at $O(D)$, since all involved tensors, including the additionally computed $\mathbf{m}$, $\mathbf{A}$, and $\mathbf{B}$, are of size $O(D)$. The commitment and proof sizes are reduced to square root and logarithmic complexities, $O(\sqrt{D})$ and $O(\log D)$ respectively, impacting the verifier's time for verifying the proof of opening and the sumcheck protocols.

7.3.2 Overhead of zkAttn. The overhead introduced by zkAttn is parameterized by K, the number of segments applied to the input. For an input of size mn in zkAttn, the total prover overhead, including both running time and memory consumption, is $O(Kmn)$ as per Section 7.3.1. The communication overhead and verifier time are $O(K\sqrt{mn})$. In comparison with the bit-decomposition method, which incurs an $\Omega(mn \log_2 B)$ overhead, zkAttn built upon tlookup achieves an $O\left(\frac{K}{\log_2 B}\right)$ reduction in prover overhead. Practically, K is chosen to be small enough to minimize both error and overhead while avoiding overly large segments (for example, $K = 1$, which would require compiling all possible input-output pairs into a table, leading to an impractical overhead of at least $\Omega(B)$ for a unrealistically large total number of possible inputs B that exceeds mn, where the previous analysis on overhead would not apply).

7.3.3 Overall overhead. zkLLM benefits from the linear prover overhead of sumcheck protocols and the logarithmic and square-root verifier overheads of sumchecks and Pedersen commitments, respectively. Specialized sumchecks for tensor operations, like matrix multiplications, achieve less complexity than the computation process itself, further reducing proof overhead for each layer. Assuming prover overhead, communication cost, and verifier overhead per layer of an L-layer LLM are $t_{\mathcal{P}}$, c, and $t_{\mathcal{V}}$ respectively, the total overheads of zkLLM scale naturally to $O(Lt_{\mathcal{P}})$, $O(Lc)$, and $O(Lt_{\mathcal{V}})$. Where the latter two can be further reduced to $O(\sqrt{Lc})$ and $O(\sqrt{L}t_{\mathcal{V}})$ by leveraging the repetitive structure across layers and batching up the commitments [29]. Additionally, unlike univariate polynomial-based ZKP systems that must be serialized, the use of sumcheck protocols over multilinear extensions and the compatible Pedersen commitment scheme in zkLLM allows for highly parallelized proof generation, thereby enabling efficient proof generation in a reasonable time.

8 EXPERIMENTS

We developed zkLLM using CUDA, basing it on the CUDA code for the BLS12-381 curve [6] produced by the ec-gpu package [16]. The implementation of sequential verifier tasks, which cannot efficiently utilize CUDA, was adapted from the zkCNN implementation [29] that relies on the mcl package [31]. We evaluated zkLLM for inferences on two classes of open-source LLMs, namely OPT [57] and LLaMa-2 [43], supporting sizes up to 13 billion parameters. For both types of models, our focus was on performing verifiable inferences using the designated models, applying samples with the default sequence length of 2048 from the C4 dataset [36]. Our experiments were conducted with resources including 124.5GB of memory, 12 CPU cores of an AMD EPYC 7413 (2.65 GHz with 128M cache L3), and an NVIDIA A100SMX4 GPU with 40GB of memory, all allocated from a computing node.

Throughout our experiments, we consistently set the scaling factor for both data embedding and model parameters at 2^{16}. All rescaling operations were integrated with the subsequent activation functions. As detailed in Section 4.2, this integration necessitated the use of multiple tlookups. Specifically, the number of tlookups corresponds to the number of times the input to the activation function requires rescaling, with each tlookup having a size of 2^{16}, to avoid excessive memory usage.

Additionally, since the input to the Softmax function in zkAttn of each layer undergoes two multiplication operations, the cumulative scaling factor reaches 2^{64}. To manage this, we deployed $K = 5$ tlookups, each of size 2^{16}. This setup includes $L = 3$ least significant segments, with the remaining two segments accommodating all potential inputs within a scale of 2^{16} and a precision of 2^{-16} when reverted to the real domain.

The tolerable error margins were selected in accordance with Section 7.1. This approach resulted in an approximate total $\mathcal{L}_1$ error of 10^{-2} on the output, a level comparable to the rounding error induced by half-precision floating points used in state-of-the-art

Table 1: The overhead of zkLLM on OPT and LLaMa-2.

Model	OPT-125M	OPT-350M	OPT-1.3B	OPT-2.7B	OPT-6.7B	OPT-13B	LLaMa-2-7B	LLaMa-2-13B
Committing time (s)	11.8	33.1	127	273	654	1.27×10^3	531	986
Commitment size (MB)	0.996	1.67	3.32	4.58	7.22	10.1	7.97	11.0
Prover time (s)	73.9	111	221	352	548	713	620	803
Proof size (kB)	141	144	147	152	157	160	183	188
Verifier time (s)	0.342	0.593	0.899	1.41	2.08	3.71	2.36	3.95
Memory usage (GB)	1.88	2.38	3.71	6.60	15.0	22.9	15.5	23.1
C4 Perplexity (orig)	26.56	22.59	16.07	14.34	12.71	12.06	7.036	6.520
C4 Perplexity (quant)	26.65	22.66	16.12	14.37	12.73	12.07	7.049	6.528

LLMs. Notably, all proofs involved in the results of this section have been successfully validated by the semi-honest verifier.

In Table 1, we present detailed information regarding the overhead associated with zkLLM for models of various sizes. The data includes the time required for the prover to commit and hide the model (committing time and commitment sizes), as well as the prover's time, proof size, and the verifier's time in response to an input prompt from the latter.

As the first endeavor to apply zero-knowledge proofs to LLMs with up to 13 billion parameters, to the best of our knowledge, zkLLM has achieved significant results toward practical and industrial applicability. Once the model is trained, the prover requires up to 20 minutes to commit to the model and subsequently publish a commitment of approximately 10MB for public scrutiny via zkLLM. When prompted by the verifier, the prover is able to generate a proof for the correctness of the entire inference process in less than 15 minutes. Additionally, as the design of zkAttn effectively resolves the inherent bottleneck of listing all input-output pairs, the memory consumption is effectively controlled under 23.1GB, which fits zkLLM into commonly used GPUs in machine learning, like Tesla V100 and A100.

It is important to note that while the time for committing generally scales with the size of the parameters, the time for generating proofs scales more slowly. This slower scaling is attributed to the less significant differences in the complexities of intermediate computations and more efficient use of parallel computing resources as the size of the tensors increases. Ultimately, the succinct proof, which is only about 100kB in size, can be verified by the verifier in a matter of seconds. Despite this efficiency, the verifier is provably unable to glean any additional information about the model or the inference result, ensuring the correctness of the inference while maintaining the confidentiality of the model parameters.

Moreover, the numerical error due to the unavoidable discretization of the entire process for the application of the cryptographic tools does not cause significant accuracy drops: on the C4 dataset [36], the increase of perplexity is less than 0.1, and the impact diminishes to less than 0.01 as the sizes of the models scale to 13B.

In Figure 4, we further compare zkLLM with zkML [26], the first zero-knowledge proof to have achieved verifiable inference for GPT-2 under identical hardware conditions. Beyond the size of GPT-2 (1.5B parameters), where zkML results in an out-of-memory (OOM) error, we provide an estimation of the required proving time. Thanks to the design tailored for non-arithmetic tensor operations and the attention mechanism prevalent in LLMs, as well as its CUDA

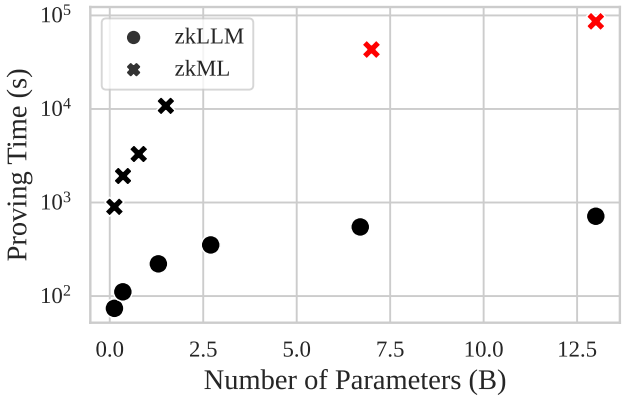

Figure 4: Comparison between zkLLM and zkML. Results of test cases with OOM errors are estimated and marked in red.

implementation, zkLLM extends the zero-knowledge verifiability to LLMs with 10x larger sizes, achieving an approximate 50x speedup.

8.1 Additional experimental results on tlookup and zkAttn

To further demonstrate the efficiency of tlookup in addressing non-arithmetic tensor operations in deep learning, as well as zkAttn, which is pivotal for verifiable computations in LLMs, we isolated an instance of zkAttn from the first layer of variously sized OPT models on input sequences of different lengths. We then measured the overhead incurred by zkAttn, including the multiple tlookups it encompasses. The results are presented in Figure 5. Note that the proof size and verifying time do not scale linearly with the number of layers, as discussed in Section 7.3.3.

It is evident that, in contrast to the overall overhead, the overhead specific to zkAttn is less influenced by the size of the model, particularly regarding proving time. However, the length of the input sequence significantly impacts various aspects of the overhead. This observation can be attributed to the fact that the Attention mechanism, while involving LLM parameters, is more significantly affected by the interactions between intermediate values (e.g., $\mathbf{Q}, \mathbf{K}, \mathbf{V}$), where their dimensions play a crucial role in determining the overhead.

Notably, the largest tested sequence length, 4096, exceeds the original design specifications of OPT models and was primarily

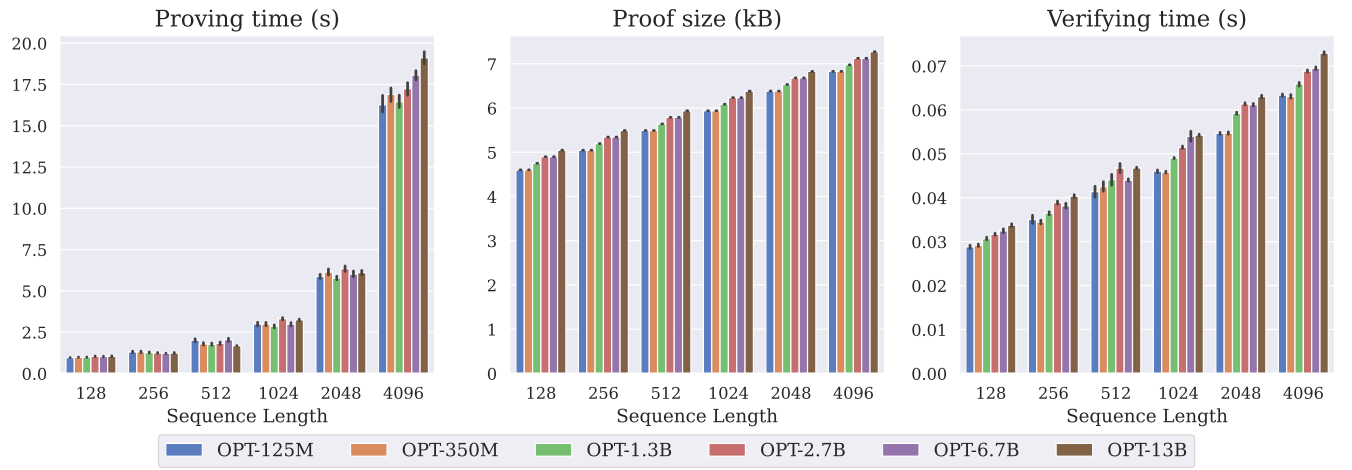

Figure 5: Overhead of zkAttn**.**

included as a reference to assess the impact of sequence length on overhead. In contrast, in Table 1, which documents overall results for zkLLM, we set the sequence length to 2048—the maximum feasible value—to maintain experimental consistency and fairness.

9 RELATED WORKS

Starting with Zhang et al. in 2020 [55], the field of zero-knowledge machine learning inference has seen active development. Initial research, parallel to the surge in computer vision studies, primarily concentrated on authenticating inference results for computer vision tasks over convolutional neural networks (CNNs). Key contributions include zkCNN [29], ZEN [15], vCNN [27], pvCNN [48], zkML [26], Mystique [47], and ezDPS [46]. These works aimed to optimize the adaptation of the entire training process to zero-knowledge proof (ZKP) backends, such as zkSNARKS [3–5, 7, 9, 19, 23, 28, 33], by leveraging the special structures of computations within CNNs. Notably, zkCNN [29] introduced a specialized interactive proof protocol for convolutional layers based on the GKR protocol [22] and its refinements [50, 51, 56]. This protocol achieved efficient proofs (less than 2 minutes) on VGG-scale CNNs, highlighting the necessity of specialized protocols for realistic zero-knowledge machine learning inference. However, as of our current knowledge, there exists a gap in zero-knowledge inferences over LLMs. The intricate structures and enormous sizes of LLMs present challenges not addressed by previous studies focused on CNNs, necessitating novel theoretical and experimental developments.

Conversely, while pioneering studies have addressed ZKPs for machine learning training, with works such as VeriML [58], proof of unlearning [14, 49], and zkPoT [20] focusing on elementary algorithms like Support Vector Machines (SVM), logistic regression, and small neural networks (up to several thousand parameters), extending zero-knowledge proofs to training LLMs may pose insurmountable challenges. The vast complexity inherent in training LLMs could render the zero-knowledge proofs for their training impractical.

10 CONCLUSION

This paper introduces zkLLM, marking the inaugural specialized zero-knowledge proof tailored for large language models, as far as our current knowledge extends. zkLLM pioneers zero-knowledge verifiable computations for general non-arithmetic operations within neural networks, ensuring full parallelizability and zero additional overhead through the implementation of tlookup. Building on this foundation, we further present zkAttn, a novel zero-knowledge proof specifically designed for the attention mechanism—a pivotal component underpinning the exceptional performance of modern LLMs. With a CUDA implementation optimized for parallel computing resources in deep learning, zkLLM achieves a groundbreaking milestone as the first study to provide zero-knowledge verifiability for LLMs with 13 billion parameters. This endeavor stands as a significant contribution towards fortifying the legitimacy of LLMs in light of their transformative impact on various domains.

ACKNOWLEDGMENTS

This work is supported by NSERC Discovery Grant RGPIN-2022-03215, DGECR-2022-00357.

REFERENCES

[1] Rohan Anil, Sebastian Borgeaud, Yonghui Wu, Jean-Baptiste Alayrac, Jiahui Yu, Radu Soricut, Johan Schalkwyk, Andrew M. Dai, Anja Hauth, Katie Millican, David Silver, Slav Petrov, Melvin Johnson, Ioannis Antonoglou, Julian Schrittwieser, Amelia Glaese, Jilin Chen, Emily Pitler, Timothy P. Lillicrap, Angeliki Lazaridou, Orhan Firat, James Molloy, Michael Isard, Paul Ronald Barham, Tom Hennigan, Benjamin Lee, Fabio Viola, Malcolm Reynolds, Yuanzhong Xu, Ryan Doherty, Eli Collins, Clemens Meyer, Eliza Rutherford, Erica Moreira, Kareem Ayoub, Megha Goel, George Tucker, Enrique Piqueras, Maxim Krikun, Iain Barr, Nikolay Savinov, Ivo Danihelka, Becca Roelofs, Anaïs White, Anders Andreassen, Tamara von Glehn, Lakshman Yagati, Mehran Kazemi, Lucas Gonzalez, Misha Khalman, Jakub Sygnowski, and et al. 2023. Gemini: A Family of Highly Capable Multimodal Models. *CoRR* abs/2312.11805 (2023). https://doi.org/10.48550/ARXIV.2312.11805 arXiv:2312.11805

[2] Lei Jimmy Ba, Jamie Ryan Kiros, and Geoffrey E. Hinton. 2016. Layer Normalization. *CoRR* abs/1607.06450 (2016). arXiv:1607.06450 http://arxiv.org/abs/1607.06450

[3] Rishabh Bhadauria, Zhiyong Fang, Carmit Hazay, Muthuramakrishnan Venkitasubramaniam, Tiancheng Xie, and Yupeng Zhang. 2020. Ligero++: A New Optimized Sublinear IOP. In *CCS '20: 2020 ACM SIGSAC Conference on Computer*

and Communications Security, Virtual Event, USA, November 9-13, 2020, Jay Ligatti, Xinming Ou, Jonathan Katz, and Giovanni Vigna (Eds.). ACM, 2025–2038. https://doi.org/10.1145/3372297.3417893

[4] Nir Bitansky, Alessandro Chiesa, Yuval Ishai, Rafail Ostrovsky, and Omer Paneth. 2022. Succinct Non-Interactive Arguments via Linear Interactive Proofs. *J. Cryptol.* 35, 3 (2022), 15. https://doi.org/10.1007/Ss00145-022-09424-4

[5] Dan Boneh, Justin Drake, Ben Fisch, and Ariel Gabizon. 2020. Halo Infinite: Recursive zk-SNARKs from any Additive Polynomial Commitment Scheme. *IACR Cryptol. ePrint Arch.* (2020), 1536. https://eprint.iacr.org/2020/1536

[6] Dan Boneh, Ben Lynn, and Hovav Shacham. 2001. Short Signatures from the Weil Pairing. In *Advances in Cryptology - ASIACRYPT 2001, 7th International Conference on the Theory and Application of Cryptology and Information Security, Gold Coast, Australia, December 9-13, 2001, Proceedings (Lecture Notes in Computer Science, Vol. 2248)*, Colin Boyd (Ed.). Springer, 514–532. https://doi.org/10.1007/3-540-45682-1_30

[7] Sean Bowe, Jack Grigg, and Daira Hopwood. 2019. Halo: Recursive Proof Composition without a Trusted Setup. *IACR Cryptol. ePrint Arch.* (2019), 1021. https://eprint.iacr.org/2019/1021

[8] Tom B. Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, Sandhini Agarwal, Ariel Herbert-Voss, Gretchen Krueger, Tom Henighan, Rewon Child, Aditya Ramesh, Daniel M. Ziegler, Jeffrey Wu, Clemens Winter, Christopher Hesse, Mark Chen, Eric Sigler, Mateusz Litwin, Scott Gray, Benjamin Chess, Jack Clark, Christopher Berner, Sam McCandlish, Alec Radford, Ilya Sutskever, and Dario Amodei. 2020. Language Models are Few-Shot Learners. In *Advances in Neural Information Processing Systems 33: Annual Conference on Neural Information Processing Systems 2020, NeurIPS 2020, December 6-12, 2020, virtual*, Hugo Larochelle, Marc'Aurelio Ranzato, Raia Hadsell, Maria-Florina Balcan, and Hsuan-Tien Lin (Eds.). https://proceedings.neurips.cc/paper/2020/hash/1457c0d6bfcb4967418bfb8ac142f64a-Abstract.html

[9] Binyi Chen, Benedikt Bünz, Dan Boneh, and Zhenfei Zhang. 2023. HyperPlonk: Plonk with Linear-Time Prover and High-Degree Custom Gates. In *Advances in Cryptology - EUROCRYPT 2023 - 42nd Annual International Conference on the Theory and Applications of Cryptographic Techniques, Lyon, France, April 23-27, 2023, Proceedings, Part II (Lecture Notes in Computer Science, Vol. 14005)*, Carmit Hazay and Martijn Stam (Eds.). Springer, 499–530. https://doi.org/10.1007/978-3-031-30617-4_17

[10] Alessandro Chiesa, Michael A. Forbes, and Nicholas Spooner. 2017. A Zero Knowledge Sumcheck and its Applications. *Electron. Colloquium Comput. Complex.* TR17-057 (2017). ECCC:TR17-057 https://eccc.weizmann.ac.il/report/2017/057

[11] Aakanksha Chowdhery, Sharan Narang, Jacob Devlin, Maarten Bosma, Gaurav Mishra, Adam Roberts, Paul Barham, Hyung Won Chung, Charles Sutton, Sebastian Gehrmann, Parker Schuh, Kensen Shi, Sasha Tsvyashchenko, Joshua Maynez, Abhishek Rao, Parker Barnes, Yi Tay, Noam Shazeer, Vinodkumar Prabhakaran, Emily Reif, Nan Du, Ben Hutchinson, Reiner Pope, James Bradbury, Jacob Austin, Michael Isard, Guy Gur-Ari, Pengcheng Yin, Toju Duke, Anselm Levskaya, Sanjay Ghemawat, Sunipa Dev, Henryk Michalewski, Xavier Garcia, Vedant Misra, Kevin Robinson, Liam Fedus, Denny Zhou, Daphne Ippolito, David Luan, Hyeontaek Lim, Barret Zoph, Alexander Spiridonov, Ryan Sepassi, David Dohan, Shivani Agrawal, Mark Omernick, Andrew M. Dai, Thanumalayan Sankaranarayana Pillai, Marie Pellat, Aitor Lewkowycz, Erica Moreira, Rewon Child, Oleksandr Polozov, Katherine Lee, Zongwei Zhou, Xuezhi Wang, Brennan Saeta, Mark Diaz, Orhan Firat, Michele Catasta, Jason Wei, Kathy Meier-Hellstern, Douglas Eck, Jeff Dean, Slav Petrov, and Noah Fiedel. 2023. PaLM: Scaling Language Modeling with Pathways. *J. Mach. Learn. Res.* 24 (2023), 240:1–240:113. http://jmlr.org/papers/v24/22-1144.html

[12] Graham Cormode, Michael Mitzenmacher, and Justin Thaler. 2012. Practical verified computation with streaming interactive proofs. In *Innovations in Theoretical Computer Science 2012, Cambridge, MA, USA, January 8-10, 2012*, Shafi Goldwasser (Ed.). ACM, 90–112. https://doi.org/10.1145/2090236.2090245

[13] Liam Eagen, Dario Fiore, and Ariel Gabizon. 2022. cq: Cached quotients for fast lookups. *IACR Cryptol. ePrint Arch.* (2022), 1763. https://eprint.iacr.org/2022/1763

[14] Thorsten Eisenhofer, Doreen Riepel, Varun Chandrasekaran, Esha Ghosh, Olga Ohrimenko, and Nicolas Papernot. 2022. Verifiable and Provably Secure Machine Unlearning. *CoRR* abs/2210.09126 (2022). https://doi.org/10.48550/arXiv.2210.09126 arXiv:2210.09126

[15] Boyuan Feng, Lianke Qin, Zhenfei Zhang, Yufei Ding, and Shumo Chu. 2021. ZEN: Efficient Zero-Knowledge Proofs for Neural Networks. *IACR Cryptol. ePrint Arch.* (2021), 87. https://eprint.iacr.org/2021/087

[16] Filecoin. 2023. ec-gpu. https://github.com/filecoin-project/ec-gpu. Accessed: 2024-01-22.

[17] Ariel Gabizon and Dmitry Khovratovich. 2022. flookup: Fractional decomposition-based lookups in quasi-linear time independent of table size. *IACR Cryptol. ePrint Arch.* (2022), 1447. https://eprint.iacr.org/2022/1447

[18] Ariel Gabizon and Zachary J. Williamson. 2020. plookup: A simplified polynomial protocol for lookup tables. *IACR Cryptol. ePrint Arch.* (2020), 315. https://eprint.iacr.org/2020/315

[19] Ariel Gabizon, Zachary J. Williamson, and Oana Ciobotaru. 2019. PLONK: Permutations over Lagrange-bases for Oecumenical Noninteractive arguments of Knowledge. *IACR Cryptol. ePrint Arch.* (2019), 953. https://eprint.iacr.org/2019/953

[20] Sanjam Garg, Aarushi Goel, Somesh Jha, Saeed Mahloujifar, Mohammad Mahmoody, Guru-Vamsi Policharla, and Mingyuan Wang. 2023. Experimenting with Zero-Knowledge Proofs of Training. *IACR Cryptol. ePrint Arch.* (2023), 1345. https://eprint.iacr.org/2023/1345

[21] Zahra Ghodsi, Tianyu Gu, and Siddharth Garg. 2017. SafetyNets: Verifiable Execution of Deep Neural Networks on an Untrusted Cloud. In *Advances in Neural Information Processing Systems 30: Annual Conference on Neural Information Processing Systems 2017, December 4-9, 2017, Long Beach, CA, USA*, Isabelle Guyon, Ulrike von Luxburg, Samy Bengio, Hanna M. Wallach, Rob Fergus, S. V. N. Vishwanathan, and Roman Garnett (Eds.). 4672–4681. https://proceedings.neurips.cc/paper/2017/hash/6048ff4e8cb07aa60b6777b6f7384d52-Abstract.html

[22] Shafi Goldwasser, Yael Tauman Kalai, and Guy N. Rothblum. 2008. Delegating computation: interactive proofs for muggles. In *Proceedings of the 40th Annual ACM Symposium on Theory of Computing, Victoria, British Columbia, Canada, May 17-20, 2008*, Cynthia Dwork (Ed.). ACM, 113–122. https://doi.org/10.1145/1374376.1374396

[23] Jens Groth. 2016. On the Size of Pairing-Based Non-interactive Arguments. In *Advances in Cryptology - EUROCRYPT 2016 - 35th Annual International Conference on the Theory and Applications of Cryptographic Techniques, Vienna, Austria, May 8-12, 2016, Proceedings, Part II (Lecture Notes in Computer Science, Vol. 9666)*, Marc Fischlin and Jean-Sébastien Coron (Eds.). Springer, 305–326. https://doi.org/10.1007/978-3-662-49896-5_11

[24] Ulrich Haböck. 2022. Multivariate lookups based on logarithmic derivatives. *IACR Cryptol. ePrint Arch.* (2022), 1530. https://eprint.iacr.org/2022/1530

[25] Dan Hendrycks and Kevin Gimpel. 2016. Bridging Nonlinearities and Stochastic Regularizers with Gaussian Error Linear Units. *CoRR* abs/1606.08415 (2016). arXiv:1606.08415 http://arxiv.org/abs/1606.08415

[26] Daniel Kang, Tatsunori Hashimoto, Ion Stoica, and Yi Sun. 2022. Scaling up Trustless DNN Inference with Zero-Knowledge Proofs. *CoRR* abs/2210.08674 (2022). https://doi.org/10.48550/arXiv.2210.08674 arXiv:2210.08674

[27] Seunghwa Lee, Hankyung Ko, Jihye Kim, and Hyunok Oh. 2020. vCNN: Verifiable Convolutional Neural Network. *IACR Cryptol. ePrint Arch.* (2020), 584. https://eprint.iacr.org/2020/584

[28] Marta Ligero, Guillermo Torres, Carles Sánchez, Katerine Díaz-Chito, Raquel Perez, and Debora Gil. 2019. Selection of Radiomics Features based on their Reproducibility. In *41st Annual International Conference of the IEEE Engineering in Medicine and Biology Society, EMBC 2019, Berlin, Germany, July 23-27, 2019*. IEEE, 403–408. https://doi.org/10.1109/EMBC.2019.8857879

[29] Tianyi Liu, Xiang Xie, and Yupeng Zhang. 2021. zkCNN: Zero Knowledge Proofs for Convolutional Neural Network Predictions and Accuracy. In *CCS '21: 2021 ACM SIGSAC Conference on Computer and Communications Security, Virtual Event, Republic of Korea, November 15 - 19, 2021*, Yongdae Kim, Jong Kim, Giovanni Vigna, and Elaine Shi (Eds.). ACM, 2968–2985. https://doi.org/10.1145/3460120.3485379

[30] Carsten Lund, Lance Fortnow, Howard J. Karloff, and Noam Nisan. 1992. Algebraic Methods for Interactive Proof Systems. *J. ACM* 39, 4 (1992), 859–868. https://doi.org/10.1145/146585.146605

[31] Shigeo Mitsunari. 2023. MCL: A portable and fast pairing-based cryptography library. https://github.com/herumi/mcl. Accessed: 2024-01-22.

[32] OpenAI. 2023. GPT-4 Technical Report. *CoRR* abs/2303.08774 (2023). https://doi.org/10.48550/ARXIV.2303.08774 arXiv:2303.08774

[33] Bryan Parno, Jon Howell, Craig Gentry, and Mariana Raykova. 2013. Pinocchio: Nearly Practical Verifiable Computation. In *2013 IEEE Symposium on Security and Privacy, SP 2013, Berkeley, CA, USA, May 19-22, 2013*. IEEE Computer Society, 238–252. https://doi.org/10.1109/SP.2013.47

[34] Torben P. Pedersen. 1991. Non-Interactive and Information-Theoretic Secure Verifiable Secret Sharing. In *Advances in Cryptology - CRYPTO '91, 11th Annual International Cryptology Conference, Santa Barbara, California, USA, August 11-15, 1991, Proceedings (Lecture Notes in Computer Science, Vol. 576)*, Joan Feigenbaum (Ed.). Springer, 129–140. https://doi.org/10.1007/3-540-46766-1_9

[35] Jim Posen and Assimakis A. Kattis. 2022. Caulk+: Table-independent lookup arguments. *IACR Cryptol. ePrint Arch.* (2022), 957. https://eprint.iacr.org/2022/957

[36] Colin Raffel, Noam Shazeer, Adam Roberts, Katherine Lee, Sharan Narang, Michael Matena, Yanqi Zhou, Wei Li, and Peter J. Liu. 2020. Exploring the Limits of Transfer Learning with a Unified Text-to-Text Transformer. *J. Mach. Learn. Res.* 21 (2020), 140:1–140:67. http://jmlr.org/papers/v21/20-074.html

[37] Jacob T. Schwartz. 1980. Fast Probabilistic Algorithms for Verification of Polynomial Identities. *J. ACM* 27, 4 (1980), 701–717. https://doi.org/10.1145/322217.322225

[38] Noam Shazeer. 2020. GLU Variants Improve Transformer. *CoRR* abs/2002.05202 (2020). arXiv:2002.05202 https://arxiv.org/abs/2002.05202

[39] Jianlin Su, Murtadha H. M. Ahmed, Yu Lu, Shengfeng Pan, Wen Bo, and Yunfeng Liu. 2024. RoFormer: Enhanced transformer with Rotary Position Embedding. *Neurocomputing* 568 (2024), 127063. https://doi.org/10.1016/J.NEUCOM.2023.127063

[40] Justin Thaler. 2013. Time-Optimal Interactive Proofs for Circuit Evaluation. In *Advances in Cryptology - CRYPTO 2013 - 33rd Annual Cryptology Conference,*

Santa Barbara, CA, USA, August 18-22, 2013. Proceedings, Part II (Lecture Notes in Computer Science, Vol. 8043), Ran Canetti and Juan A. Garay (Eds.). Springer, 71–89. https://doi.org/10.1007/978-3-642-40084-1_5

[41] Justin Thaler. 2022. Proofs, Arguments, and Zero-Knowledge. *Found. Trends Priv. Secur.* 4, 2-4 (2022), 117–660. https://doi.org/10.1561/3300000030

[42] Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, Timothée Lacroix, Baptiste Rozière, Naman Goyal, Eric Hambro, Faisal Azhar, Aurélien Rodriguez, Armand Joulin, Edouard Grave, and Guillaume Lample. 2023. LLaMA: Open and Efficient Foundation Language Models. *CoRR* abs/2302.13971 (2023). https://doi.org/10.48550/ARXIV.2302.13971 arXiv:2302.13971

[43] Hugo Touvron, Louis Martin, Kevin Stone, Peter Albert, Amjad Almahairi, Yasmine Babaei, Nikolay Bashlykov, Soumya Batra, Prajjwal Bhargava, Shruti Bhosale, Dan Bikel, Lukas Blecher, Cristian Canton-Ferrer, Moya Chen, Guillem Cucurull, David Esiobu, Jude Fernandes, Jeremy Fu, Wenyin Fu, Brian Fuller, Cynthia Gao, Vedanuj Goswami, Naman Goyal, Anthony Hartshorn, Saghar Hosseini, Rui Hou, Hakan Inan, Marcin Kardas, Viktor Kerkez, Madian Khabsa, Isabel Kloumann, Artem Korenev, Punit Singh Koura, Marie-Anne Lachaux, Thibaut Lavril, Jenya Lee, Diana Liskovich, Yinghai Lu, Yuning Mao, Xavier Martinet, Todor Mihaylov, Pushkar Mishra, Igor Molybog, Yixin Nie, Andrew Poulton, Jeremy Reizenstein, Rashi Rungta, Kalyan Saladi, Alan Schelten, Ruan Silva, Eric Michael Smith, Ranjan Subramanian, Xiaoqing Ellen Tan, Binh Tang, Ross Taylor, Adina Williams, Jian Xiang Kuan, Puxin Xu, Zheng Yan, Iliyan Zarov, Yuchen Zhang, Angela Fan, Melanie Kambadur, Sharan Narang, Aurélien Rodriguez, Robert Stojnic, Sergey Edunov, and Thomas Scialom. 2023. Llama 2: Open Foundation and Fine-Tuned Chat Models. *CoRR* abs/2307.09288 (2023). https://doi.org/10.48550/ARXIV.2307.09288 arXiv:2307.09288

[44] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. 2017. Attention is All you Need. In *Advances in Neural Information Processing Systems 30: Annual Conference on Neural Information Processing Systems 2017, December 4-9, 2017, Long Beach, CA, USA*, Isabelle Guyon, Ulrike von Luxburg, Samy Bengio, Hanna M. Wallach, Rob Fergus, S. V. N. Vishwanathan, and Roman Garnett (Eds.). 5998–6008. https://proceedings.neurips.cc/paper/2017/hash/3f5ee243547dee91fbd053c1c4a845aa-Abstract.html

[45] Riad S. Wahby, Ioanna Tzialla, Abhi Shelat, Justin Thaler, and Michael Walfish. 2018. Doubly-Efficient zkSNARKs Without Trusted Setup. In *2018 IEEE Symposium on Security and Privacy, SP 2018, Proceedings, 21-23 May 2018, San Francisco, California, USA*. IEEE Computer Society, 926–943. https://doi.org/10.1109/SP.2018.00060

[46] Haodi Wang and Thang Hoang. 2023. ezDPS: An Efficient and Zero-Knowledge Machine Learning Inference Pipeline. *Proc. Priv. Enhancing Technol.* 2023, 2 (2023), 430–448. https://doi.org/10.56553/popets-2023-0061

[47] Chenkai Weng, Kang Yang, Xiang Xie, Jonathan Katz, and Xiao Wang. 2021. Mystique: Efficient Conversions for Zero-Knowledge Proofs with Applications to Machine Learning. In *30th USENIX Security Symposium, USENIX Security 2021, August 11-13, 2021*, Michael Bailey and Rachel Greenstadt (Eds.). USENIX Association, 501–518. https://www.usenix.org/conference/usenixsecurity21/presentation/weng

[48] Jia-Si Weng, Jian Weng, Gui Tang, Anjia Yang, Ming Li, and Jia-Nan Liu. 2023. pvCNN: Privacy-Preserving and Verifiable Convolutional Neural Network Testing. *IEEE Trans. Inf. Forensics Secur.* 18 (2023), 2218–2233. https://doi.org/10.1109/TIFS.2023.3262932

[49] Jiasi Weng, Shenglong Yao, Yuefeng Du, Junjie Huang, Jian Weng, and Cong Wang. 2022. Proof of Unlearning: Definitions and Instantiation. *CoRR* abs/2210.11334 (2022). https://doi.org/10.48550/ARXIV.2210.11334 arXiv:2210.11334

[50] Tiancheng Xie, Jiaheng Zhang, Yupeng Zhang, Charalampos Papamanthou, and Dawn Song. 2019. Libra: Succinct Zero-Knowledge Proofs with Optimal Prover Computation. In *Advances in Cryptology - CRYPTO 2019 - 39th Annual International Cryptology Conference, Santa Barbara, CA, USA, August 18-22, 2019, Proceedings, Part III (Lecture Notes in Computer Science, Vol. 11694)*, Alexandra Boldyreva and Daniele Micciancio (Eds.). Springer, 733–764. https://doi.org/10.1007/978-3-030-26954-8_24

[51] Tiancheng Xie, Yupeng Zhang, and Dawn Song. 2022. Orion: Zero Knowledge Proof with Linear Prover Time. In *Advances in Cryptology - CRYPTO 2022 - 42nd Annual International Cryptology Conference, CRYPTO 2022, Santa Barbara, CA, USA, August 15-18, 2022, Proceedings, Part IV (Lecture Notes in Computer Science, Vol. 13510)*, Yevgeniy Dodis and Thomas Shrimpton (Eds.). Springer, 299–328. https://doi.org/10.1007/978-3-031-15985-5_11

[52] Arantxa Zapico, Vitalik Buterin, Dmitry Khovratovich, Mary Maller, Anca Nitulescu, and Mark Simkin. 2022. Caulk: Lookup Arguments in Sublinear Time. In *Proceedings of the 2022 ACM SIGSAC Conference on Computer and Communications Security, CCS 2022, Los Angeles, CA, USA, November 7-11, 2022*, Heng Yin, Angelos Stavrou, Cas Cremers, and Elaine Shi (Eds.). ACM, 3121–3134. https://doi.org/10.1145/3548606.3560646

[53] Arantxa Zapico, Ariel Gabizon, Dmitry Khovratovich, Mary Maller, and Carla Ràfols. 2022. Baloo: Nearly Optimal Lookup Arguments. *IACR Cryptol. ePrint Arch.* (2022), 1565. https://eprint.iacr.org/2022/1565

[54] Biao Zhang and Rico Sennrich. 2019. Root Mean Square Layer Normalization. In *Advances in Neural Information Processing Systems 32: Annual Conference on Neural Information Processing Systems 2019, NeurIPS 2019, December 8-14, 2019, Vancouver, BC, Canada*, Hanna M. Wallach, Hugo Larochelle, Alina Beygelzimer, Florence d'Alché-Buc, Emily B. Fox, and Roman Garnett (Eds.). 12360–12371. https://proceedings.neurips.cc/paper/2019/hash/1e8a19426224ca89e83cef47f1e7f53b-Abstract.html

[55] Jiaheng Zhang, Zhiyong Fang, Yupeng Zhang, and Dawn Song. 2020. Zero Knowledge Proofs for Decision Tree Predictions and Accuracy. In *CCS '20: 2020 ACM SIGSAC Conference on Computer and Communications Security, Virtual Event, USA, November 9-13, 2020*, Jay Ligatti, Xinming Ou, Jonathan Katz, and Giovanni Vigna (Eds.). ACM, 2039–2053. https://doi.org/10.1145/3372297.3417278

[56] Jiaheng Zhang, Tianyi Liu, Weijie Wang, Yinuo Zhang, Dawn Song, Xiang Xie, and Yupeng Zhang. 2021. Doubly Efficient Interactive Proofs for General Arithmetic Circuits with Linear Prover Time. In *CCS '21: 2021 ACM SIGSAC Conference on Computer and Communications Security, Virtual Event, Republic of Korea, November 15 - 19, 2021*, Yongdae Kim, Jong Kim, Giovanni Vigna, and Elaine Shi (Eds.). ACM, 159–177. https://doi.org/10.1145/3460120.3484767

[57] Susan Zhang, Stephen Roller, Naman Goyal, Mikel Artetxe, Moya Chen, Shuohui Chen, Christopher Dewan, Mona T. Diab, Xian Li, Xi Victoria Lin, Todor Mihaylov, Myle Ott, Sam Shleifer, Kurt Shuster, Daniel Simig, Punit Singh Koura, Anjali Sridhar, Tianlu Wang, and Luke Zettlemoyer. 2022. OPT: Open Pre-trained Transformer Language Models. *CoRR* abs/2205.01068 (2022). https://doi.org/10.48550/ARXIV.2205.01068 arXiv:2205.01068

[58] Lingchen Zhao, Qian Wang, Cong Wang, Qi Li, Chao Shen, and Bo Feng. 2021. VeriML: Enabling Integrity Assurances and Fair Payments for Machine Learning as a Service. *IEEE Trans. Parallel Distributed Syst.* 32, 10 (2021), 2524–2540. https://doi.org/10.1109/TPDS.2021.3068195

[59] Richard Zippel. 1979. Probabilistic algorithms for sparse polynomials. In *Symbolic and Algebraic Computation, EUROSAM '79, An International Symposiumon Symbolic and Algebraic Computation, Marseille, France, June 1979, Proceedings (Lecture Notes in Computer Science, Vol. 72)*, Edward W. Ng (Ed.). Springer, 216–226. https://doi.org/10.1007/3-540-09519-5_73

A LONG VERSION

The long version of the paper is available at https://arxiv.org/abs/2404.16109.

B CODE

Our open-source implementation is available at https://github.com/jvhs0706/zkllm-ccs2024.